前段 minigame/src/main.ts:1 至 minigame/src/main.ts:59

```
/**
 * Mini-game entry point: boots the adapter, sets up the main canvas,
 * routes touch events into the UI framework, and runs the main loop.
 */
import './platform-bootstrap';
import { loadPixelFont, getMainCanvas } from './adapter';
import { Router } from './ui/framework';
import { TouchRouting } from './ui/touch-routing';
import { lobbyState } from './lobby-state';
import { LobbyScreen } from './ui/lobby';
import { DeckBuilderScreen } from './ui/deck-builder';
import { ShopScreen } from './ui/shop';
import { BattleScreen } from './ui/battle';
import { preloadAllCardArt } from './ui/card-render';
declare const tt: any;
/**
 * CDN base URL for art assets.
 *
 * 美术资源（约 26MB WebP）不上传主包，运行时从此 CDN 地址下载。
 * 上线前把 minigame-cdn/ 目录上传到你的 CDN，然后把下面的地址改成
 * 你的 CDN 地址（以 / 结尾）。
 *
 * 留空则从本地包内加载（仅开发调试用，需要把 art 目录也放进包）。
 */
const ART_CDN_BASE = 'https://cdn.example.com/pixel-frontline/';
/**
 * BGM 等大音频文件的 CDN 地址。与美术资源同一个 CDN 即可，
 * 把 minigame-cdn/audio/ 目录上传到 CDN 后自动生效。
 */
const AUDIO_CDN_BASE = ART_CDN_BASE;
export function boot(): void {
  lobbyState.initialize();
  // Route art loads to the CDN (WebP versions).
  if (ART_CDN_BASE) {
    (globalThis as { __ART_CDN_BASE__?: string }).__ART_CDN_BASE__ = ART_CDN_BASE;
  }
  if (AUDIO_CDN_BASE) {
    (globalThis as { __AUDIO_CDN_BASE__?: string }).__AUDIO_CDN_BASE__ = AUDIO_CDN_BASE;
  }
  const fontFamily = loadPixelFont();
  const canvas = getMainCanvas();
  const ctx = canvas.getContext('2d') as CanvasRenderingContext2D;
  const info = tt.getSystemInfoSync();
  const screenW = info.windowWidth;
  const screenH = info.windowHeight;
  const pixelRatio = info.pixelRatio || 1;
  ctx.scale(pixelRatio, pixelRatio);
  // ── Router + screens ──
  const router = new Router();
  const lobby = new LobbyScreen(screenW, screenH, router);
```

前段 minigame/src/main.ts:60 至 minigame/src/main.ts:116

```
const builder = new DeckBuilderScreen(screenW, screenH, router);
const shop = new ShopScreen(screenW, screenH, router);
const battle = new BattleScreen(screenW, screenH, router);
router.register('lobby', lobby);
router.register('deck-builder', builder);
router.register('shop', shop);
router.register('battle', battle);
lobbyState.onBattle = () => {
  battle.configure(lobbyState.match!);
  router.navigate('battle');
};
router.navigate('lobby');
// ── Touch routing ──
const touches = new TouchRouting((event) => router.handleTouch(event));
tt.onTouchStart((e: any) => touches.dispatch('down', e));
tt.onTouchMove((e: any) => touches.dispatch('move', e));
tt.onTouchEnd((e: any) => touches.dispatch('up', e));
tt.onTouchCancel((e: any) => touches.dispatch('cancel', e));
// ── Main loop ──
let last = Date.now();
const loop = () => {
  const now = Date.now();
  const dt = Math.min(0.1, (now - last) / 1000);
  last = now;
  router.update(dt);
  ctx.clearRect(0, 0, screenW, screenH);
  router.draw(ctx);
  requestAnimationFrame(loop);
};
requestAnimationFrame(loop);
// ── Background preload of card art ──
preloadAllCardArt();
// ── Lifecycle: pause on hide ──
if (typeof tt.onHide === 'function') {
  tt.onHide(() => {
    touches.cancelAll();
    router.active?.cancelTouches();
    battle.onAppHide();
  });
}
if (typeof tt.onShow === 'function') {
  tt.onShow(() => {
    battle.onAppShow();
  });
}
if (typeof tt.onAudioInterruptionBegin === 'function') {
  tt.onAudioInterruptionBegin(() => {
    battle.onAudioInterrupt(true);
  });
}
```

前段 minigame/src/main.ts:117 至 app/battle.tsx:37

```
  if (typeof tt.onAudioInterruptionEnd === 'function') {
    tt.onAudioInterruptionEnd(() => {
      battle.onAudioInterrupt(false);
    });
  }
  // Keep a reference so the font family is not tree-shaken away.
  void fontFamily;
}
boot();
import { createRoot } from 'react-dom/client';
import Home from '../app/page';
import '../app/globals.css';
createRoot(document.getElementById('root')!).render(<Home />);
'use client';
import {BattleFrameBudget} from '../game/battle-frame-budget';
import ContactMapMarks from './contact-map-marks';
/* eslint-disable jsx-a11y/prefer-tag-over-role -- Custom battlefield meters and the draggable mini-map have explicit accessible roles and keyboard support. */
import {
  useCallback,
  useMemo,
  useEffect,
  useRef,
  useState,
  type CSSProperties,
} from 'react';
import { createPortal } from 'react-dom';
import {
  ArrowRight,
  BookOpen,
  ChevronRight,
  Crosshair,
  Binoculars,
  Flag,
  Layers3,
  Pause,
  Play,
  Radio,
  RotateCcw,
  Shield,
  Sparkles,
  Volume2,
  VolumeX,
  X,
  Zap,
} from 'lucide-react';
import { RadioGroup, RadioGroupItem } from '@/components/ui/radio-group';
import {
  Dialog,
  DialogContent,
  DialogDescription,
```

前段 app/battle.tsx:38 至 app/battle.tsx:87

```
  DialogTitle,
} from '@/components/ui/dialog';
import {
  CARDS,
  chooseAiDeck,
  needsTarget,
  createGame, toggleRadar,
  DURATION,
  MAX_HP,
  playCard,
  playLockoutMessage,
  requestDraw,
  DRAW_TIME,
  snapshot,
  startGame,
  setOrder,
  W,
  VIEW_W,
  H,
  type Order,
  type CardId,
  type GameState,
  type HandCard,
} from '@/game/engine';
import { CardFace } from '@/game/card-art';
import { advanceBattleFrame } from '@/game/battle-clock';
import { CARD_COPY } from '@/game/card-copy';
import { render } from '@/game/render';
import { loadBattleArt, type Art } from '@/game/art';
import { watchBattleArtProgress, type BattleArtProgress } from '@/game/battle-art-baked';
import {
  DEFAULT_AUDIO,
  getBattleAudio,
  type BattleAudio,
  type AudioSettings,
} from '@/game/audio';
import { assetUrl } from '@/game/asset-url';
import { DEFAULT_MAP, type MapId } from '@/game/maps';
import {
  pickSquad,
  setSquadOrder,
  selectUnitGroup,
  ordersForUnit,
  automaticFireSupport,
  trenchConstructionLabel,
} from '@/game/squad-orders';
import { unitSelectionBounds } from '@/game/selection-render';
import SquadMenu from './squad-menu';
import UnitLogisticsFan from './unit-logistics-fan';
import {vehicleOutOfFuel} from '@/game/vehicle-logistics';
```

前段 app/battle.tsx:88 至 app/battle.tsx:138

```
import {isImmobilized} from '@/game/vehicle-damage';
import {logisticsAlert,issueLogisticsOrder} from '@/game/logistics-orders';
import {logisticsIndicator,pickLogisticsIndicator} from '@/game/logistics-indicator';
import { DEFAULT_DIFFICULTY, type Difficulty } from '@/game/economy';
import {
  MISSIONS,
  missionById,
  missionDeckName,
  type MissionId,
} from '@/game/campaign';
import { createCampaignGame } from '@/game/campaign-game';
import CampaignDialogue from './campaign-dialogue';
import { DIFFICULTY_LABEL, DIFFICULTY_BONUS } from './difficulty-selector';
const timeString = (t: number) =>
  `${String(Math.floor(t / 60)).padStart(2, '0')}:${String(Math.floor(t % 60)).padStart(2, '0')}`;
type CardGesture = {
  uid: number;
  pointer: number;
  startX: number;
  startY: number;
  centerX: number;
  centerY: number;
  width: number;
  dragging: boolean;
  bounds: { left: number; right: number; top: number; bottom: number };
};
function dragPosition(gesture: CardGesture, clientX: number, clientY: number) {
  const x = gesture.centerX + clientX - gesture.startX;
  const y = gesture.centerY + clientY - gesture.startY;
  const { left, right, top, bottom } = gesture.bounds;
  return {
    uid: gesture.uid,
    x,
    y,
    width: gesture.width,
    outside: x < left || x > right || y < top || y > bottom,
  };
}
export default function Battle({
  playerDeck,
  aiDeck,
  seed,
  mapId = DEFAULT_MAP,
  difficulty = DEFAULT_DIFFICULTY,
  night = false,
  missionId,
  onMissionComplete,
  onNextMission,
  onExit,
}: {
```

前段 app/battle.tsx:139 至 app/battle.tsx:188

```
  playerDeck: CardId[];
  aiDeck: CardId[];
  seed: number;
  mapId?: MapId;
  difficulty?: Difficulty;
  night?: boolean;
  missionId?: MissionId;
  onMissionComplete?: (id: MissionId) => void;
  onNextMission?: () => void;
  onExit: () => void;
}) {
  const [initialGame] = useState(() =>
    missionId
      ? createCampaignGame(seed, playerDeck, missionId, difficulty)
      : createGame(seed, playerDeck, aiDeck, mapId, { difficulty, night }),
  );
  const game = useRef<GameState>(initialGame);
  const [view, setView] = useState(() => snapshot(initialGame));
  const completionRecorded = useRef(false);
  const mission = missionId ? missionById(missionId) : null;
  const [dialogueOpen, setDialogueOpen] = useState(!!missionId);
  const dialogueOpenRef = useRef(!!missionId),
    dialogueResume = useRef(true);
  const timeLimit = view.campaign?.duration ?? DURATION;
  const finalMission = mission?.id === MISSIONS[MISSIONS.length - 1].id;
  const captureStages = mission?.capturePoints?.length ?? 1;
  const captureRequired = view.campaign?.captureRequired ?? 15;
  const missionStatus =
    mission?.objective === 'capture'
      ? `${captureStages > 1 ? `${(view.campaign?.objectiveIndex ?? 0) + 1}/${captureStages} · ` : ''}${view.campaign?.objectiveLabel ?? '电台'} ${Math.floor(view.campaign?.captureProgress ?? 0)} / ${captureRequired}秒`
      : mission?.objective === 'defend'
        ? '守住己方指挥部'
        : '摧毁敌方指挥部';
  useEffect(() => {
    if (
      missionId &&
      view.status === 'finished' &&
      view.result === 0 &&
      !completionRecorded.current
    ) {
      completionRecorded.current = true;
      onMissionComplete?.(missionId);
    }
  }, [missionId, view.status, view.result, onMissionComplete]);
  const [selected, setSelected] = useState<number | null>(null);
  const [selectedSquad, setSelectedSquad] = useState<number | null>(null);
  const selectedSquadRef = useRef<number | null>(null);
  const [logisticsUid,setLogisticsUid]=useState<number|null>(null);
  const [logisticsAnchor,setLogisticsAnchor]=useState({x:0,y:0});
  const openLogistics=useCallback((uid:number)=>{
```

前段 app/battle.tsx:189 至 app/battle.tsx:238

```
    const u=game.current.units.find(v=>v.uid===uid);
    const mark=u&&logisticsIndicator(u);
    if(!mark)return;
    setLogisticsUid(uid);setLogisticsAnchor({x:mark.x,y:mark.y});
    selectedSquadRef.current=null;setSelectedSquad(null);
  },[]);
  const [squadMenuAnchor, setSquadMenuAnchor] = useState<{ x: number; y: number } | null>(null);
  const selectSquad = useCallback((id: number | null) => {
    selectedSquadRef.current = id;
    setSelectedSquad(id);
    const members = game.current.units.filter((u) => u.side === 0 && u.squad === id &&
      u.hp > 0 && !u.surrendered && !u.wounded && !u.rappelling);
    // Keep the command fan at the place it was opened. Stance changes and
    // regrouping must not move a button between pointer-down and its click.
    setSquadMenuAnchor(members.length ? {
      x: members.reduce((sum, u) => sum + u.x, 0) / members.length,
      y: CARDS[members[0].id].air
        ? Math.max(...members.map((u) => u.y)) + 140
        : Math.min(...members.map((u) => unitSelectionBounds(u).y)) - 18,
    } : null);
  }, []);
  const selectedRef = useRef<number | null>(null);
  const [panel, setPanel] = useState<'guide' | 'deck' | 'card' | null>(null);
  const [inspectUid, setInspectUid] = useState<number | null>(null);
  const cardHold = useRef<{
    uid: number;
    pointer: number;
    x: number;
    y: number;
    timer: ReturnType<typeof setTimeout>;
  } | null>(null);
  const heldClick = useRef<number | null>(null);
  const handArea = useRef<HTMLDivElement>(null);
  const cardGesture = useRef<CardGesture | null>(null);
  const [draggedCard, setDraggedCard] = useState<{
    uid: number;
    x: number;
    y: number;
    width: number;
    outside: boolean;
  } | null>(null);
  const portraitGate = useRef(false);
  const panelPause = useRef(false);
  const [audioSettings, setAudioSettings] = useState({ ...DEFAULT_AUDIO });
  const sound = audioSettings.enabled;
  const [assetsReady, setAssetsReady] = useState(false);
  const [assetError, setAssetError] = useState(false);
  const [assetAttempt, setAssetAttempt] = useState(0);
  const [assetProgress, setAssetProgress] = useState<BattleArtProgress>({stage: 'download', completed: 0, total: 2});
  const [assetLoadMs, setAssetLoadMs] = useState<number | null>(null);
```

前段 app/battle.tsx:239 至 app/battle.tsx:288

```
  const [message, setMessage] = useState('');
  const messageTimer = useRef<ReturnType<typeof setTimeout> | null>(null);
  const viewport = useRef(VIEW_W);
  const [viewportWidth, setViewportWidth] = useState(VIEW_W);
  const [touchMode, setTouchMode] = useState(false);
  const mapPointer = useRef<number | null>(null);
  const camera = useRef(initialGame.campaign?.initialCamera ?? 0),
    [cameraView, setCameraView] = useState(
      initialGame.campaign?.initialCamera ?? 0,
    ),
    dragView = useRef<{
      x: number;
      y: number;
      camera: number;
      id: number;
    } | null>(null),
    didDrag = useRef(false),
    keys = useRef(new Set<string>());
  const moveCamera = useCallback((x: number) => {
    camera.current = Math.max(0, Math.min(W - viewport.current, x));
    // The canvas reads this ref on every animation frame. Let the existing HUD
    // refresh publish its position instead of rebuilding the whole hand for
    // every pointer sample during a drag.
  }, []);
  const canvas = useRef<HTMLCanvasElement>(null),
    art = useRef<Art | null>(null),
    hover = useRef<number | null>(null),
    pointerScreen = useRef<number | null>(null),
    audio = useRef<BattleAudio | null>(null);
  useEffect(() => {
    const mixer = getBattleAudio();
    audio.current = mixer;
    // 进入战斗立即激活音乐（菜单已解锁时无缝延续，未解锁时在首次手势后播放）。
    mixer.setActive(true);
    void mixer.unlock();
    // 轻量双保险：进战斗即推音乐，300ms 后再试一次覆盖异步解锁的空窗。
    mixer.kick();
    const kickTimer = setTimeout(() => mixer.kick(), 300);
    let mounted = true;
    queueMicrotask(() => {
      if (mounted) setAudioSettings({ ...mixer.settings });
    });
    const unlock = () => {
      void mixer.unlock();
    };
    window.addEventListener('pointerdown', unlock);
    window.addEventListener('keydown', unlock);
    return () => {
      mounted = false;
      clearTimeout(kickTimer);
```

前段 app/battle.tsx:289 至 app/battle.tsx:338

```
      window.removeEventListener('pointerdown', unlock);
      window.removeEventListener('keydown', unlock);
      mixer.setActive(false);
    };
  }, []);
  const refresh = useCallback(() => setView(snapshot(game.current!)), []);
  const toast = useCallback((text: string) => {
    setMessage(text);
    if (messageTimer.current) clearTimeout(messageTimer.current);
    messageTimer.current = setTimeout(() => setMessage(''), 2800);
  }, []);
  const choose = useCallback(
    (uid: number | null) => {
      if (uid !== null) {selectSquad(null);setLogisticsUid(null);}
      selectedRef.current = uid;
      setSelected(uid);
      pointerScreen.current = null;
      const h = game.current.players[0].hand.find((h) => h.uid === uid);
      hover.current =
        h && needsTarget(h.id) ? camera.current + viewport.current * 0.6 : null;
    },
    [selectSquad],
  );
  const execute = useCallback(
    (uid: number, x?: number) => {
      const h = game.current.players[0].hand.find((h) => h.uid === uid);
      const entryOrTarget = h && needsTarget(h.id) ? x : undefined;
      const result = playCard(game.current!, 0, uid, entryOrTarget);
      if (result.ok) choose(null);
      else toast(result.message);
      refresh();
      return result;
    },
    [choose, refresh, toast],
  );
  const drawCard = useCallback(() => {
    const result = requestDraw(game.current!, 0);
    toast(result.message);
    refresh();
    return result;
  }, [refresh, toast]);
  const cancelCardHold = useCallback(() => {
    if (cardHold.current) clearTimeout(cardHold.current.timer);
    cardHold.current = null;
  }, []);
  const interruptCardHold = useCallback(() => {
    const uid = cardGesture.current?.uid ?? cardHold.current?.uid;
    if (uid !== undefined) heldClick.current = uid;
    cardGesture.current = null;
    setDraggedCard(null);
```

前段 app/battle.tsx:339 至 app/battle.tsx:388

```
    cancelCardHold();
  }, [cancelCardHold]);
  const start = useCallback(() => {
    if (!art.current) {
      toast('美术资源正在加载，请稍候');
      return;
    }
    startGame(game.current!);
    if (dialogueOpenRef.current) game.current.status = 'paused';
    refresh();
  }, [refresh, toast]);
  const reset = useCallback(() => {
    interruptCardHold();
    const nextSeed = Date.now();
    selectSquad(null);
    completionRecorded.current = false;
    game.current = missionId
      ? createCampaignGame(nextSeed, playerDeck, missionId, difficulty)
      : createGame(nextSeed, playerDeck, chooseAiDeck(nextSeed), mapId, {
          difficulty,
          night,
        });
    startGame(game.current);
    if (missionId) {
      dialogueResume.current = true;
      dialogueOpenRef.current = true;
      setDialogueOpen(true);
      game.current.status = 'paused';
    }
    choose(null);
    hover.current = null;
    camera.current = game.current.campaign?.initialCamera ?? 0;
    setCameraView(camera.current);
    setMessage('');
    refresh();
  }, [
    choose,
    refresh,
    playerDeck,
    mapId,
    difficulty,
    night,
    missionId,
    interruptCardHold,
    selectSquad,
  ]);
  const pause = useCallback(() => {
    if (dialogueOpenRef.current) return;
    interruptCardHold();
    const s = game.current!;
```

前段 app/battle.tsx:389 至 app/battle.tsx:438

```
    if (s.status === 'playing') s.status = 'paused';
    else if (s.status === 'paused') s.status = 'playing';
    refresh();
  }, [refresh, interruptCardHold]);
  const openDialogue = () => {
    if (!missionId || dialogueOpenRef.current) return;
    interruptCardHold();
    keys.current.clear();
    dialogueResume.current =
      game.current.status === 'playing' || game.current.status === 'ready';
    if (game.current.status === 'playing') game.current.status = 'paused';
    dialogueOpenRef.current = true;
    setDialogueOpen(true);
    refresh();
  };
  const closeDialogue = () => {
    dialogueOpenRef.current = false;
    setDialogueOpen(false);
    if (dialogueResume.current && art.current) {
      if (game.current.status === 'ready') startGame(game.current);
      else if (game.current.status === 'paused')
        game.current.status = 'playing';
    }
    refresh();
  };
  const openPanel = (p: 'guide' | 'deck' | 'card') => {
    interruptCardHold();
    panelPause.current = game.current!.status === 'playing';
    if (panelPause.current) game.current!.status = 'paused';
    setPanel(p);
    refresh();
  };
  const closePanel = () => {
    setPanel(null);
    setInspectUid(null);
    if (panelPause.current && game.current!.status === 'paused')
      game.current!.status = 'playing';
    panelPause.current = false;
    refresh();
  };
  const inspectHandCard = (uid: number) => {
    setInspectUid(uid);
    openPanel('card');
  };
  useEffect(() => {
    const cancel = () => interruptCardHold();
    window.addEventListener('blur', cancel);
    document.addEventListener('visibilitychange', cancel);
    return () => {
      cancelCardHold();
```

前段 app/battle.tsx:439 至 app/battle.tsx:488

```
    window.removeEventListener('blur', cancel);
    document.removeEventListener('visibilitychange', cancel);
  };
}, [cancelCardHold, interruptCardHold]);
useEffect(() => {
  const el = canvas.current!;
  const resize = () => {
    const box = el.parentElement!.getBoundingClientRect();
    if (box.width <= 0 || box.height <= 0) return;
    const next = Math.min(
      W,
      Math.max(240, Math.round((H * box.width) / box.height)),
    );
    const old = viewport.current;
    viewport.current = next;
    setViewportWidth(next);
    camera.current = Math.max(
      0,
      Math.min(
        W - next,
        camera.current === 0
          ? 0
          : game.current.campaign
            ? camera.current
            : camera.current + (old - next) / 2,
      ),
    );
    setCameraView(camera.current);
    pointerScreen.current = null;
    portraitGate.current =
      window.matchMedia('(pointer: coarse)').matches &&
      window.innerHeight > window.innerWidth;
    dragView.current = null;
    didDrag.current = false;
    hover.current = null;
    interruptCardHold();
  };
  const observer = new ResizeObserver(resize);
  observer.observe(el.parentElement!);
  resize();
  const query = window.matchMedia('(pointer: coarse)');
  const shortLandscape = window.matchMedia(
    '(max-height: 540px) and (orientation: landscape)',
  );
  const update = () => setTouchMode(query.matches || shortLandscape.matches);
  update();
  query.addEventListener('change', update);
  shortLandscape.addEventListener('change', update);
  return () => {
    observer.disconnect();
```

前段 app/battle.tsx:489 至 app/battle.tsx:538

```
    query.removeEventListener('change', update);
    shortLandscape.removeEventListener('change', update);
  };
}, [interruptCardHold]);
useEffect(() => {
  let stopped = false;
  const loadedAt = performance.now();
  setAssetError(false);
  const unwatch = watchBattleArtProgress(setAssetProgress);
  void loadBattleArt(game.current.mapId)
    .then((a) => {
      if (!stopped) {
        art.current = a;
        setAssetLoadMs(Math.round(performance.now() - loadedAt));
        setAssetsReady(true);
        startGame(game.current!);
        if (dialogueOpenRef.current) game.current.status = 'paused';
        refresh();
      }
    })
    .catch(() => {
      if (!stopped) setAssetError(true);
    });
  const ctx = canvas.current!.getContext('2d')!;
  let raf = 0,
    lastView = 0,
    accumulator = 0;
  const reduced = window.matchMedia(
    '(prefers-reduced-motion: reduce)',
  ).matches;
  const budget = new BattleFrameBudget();
  const mobileDisplay = window.matchMedia('(max-width: 900px), (pointer: coarse)');
  const frame = (now: number) => {
    if (stopped) return;
    if(document.hidden || portraitGate.current) {
      budget.reset(now); accumulator = 0;
      raf = requestAnimationFrame(frame); return;
    }
    const dt = budget.take(now, mobileDisplay.matches);
    if(dt === null){raf = requestAnimationFrame(frame); return;}
    const workStarted = performance.now();
    const s = game.current!;
    if (keys.current.has('a'))
      camera.current = Math.max(0, camera.current - dt * 650);
    if (keys.current.has('d'))
      camera.current = Math.min(
        W - viewport.current,
        camera.current + dt * 650,
      );
    if (
```

前段 app/battle.tsx:539 至 app/battle.tsx:588

```
        !document.hidden &&
        !portraitGate.current &&
        !dialogueOpenRef.current &&
        s.status === 'playing'
      ) {
        accumulator = advanceBattleFrame(s, dt, accumulator);
      } else accumulator = 0;
      const card = s.players[0].hand.find((h) => h.uid === selectedRef.current);
      if (pointerScreen.current !== null && selectedRef.current !== null)
        hover.current = Math.max(
          0,
          Math.min(W, camera.current + pointerScreen.current),
        );
      if (art.current)
        render(
          ctx,
          s,
          art.current,
          card?.id ?? null,
          hover.current,
          reduced,
          camera.current,
          viewport.current,
          selectedSquadRef.current,
          mobileDisplay.matches || budget.fps === 20,
        );
      audio.current?.update(
        s,
        camera.current,
        viewport.current,
        // v99: music starts the moment the battle screen appears (status
        // 'ready'), not only after the first card is deployed.
        !document.hidden &&
          !portraitGate.current &&
          (s.status === 'playing' || s.status === 'ready'),
      );
      if (now - lastView > 90) {
        setView(snapshot(s));
        setCameraView(camera.current);
        lastView = now;
      }
      budget.record(now, performance.now() - workStarted);
      raf = requestAnimationFrame(frame);
    };
    raf = requestAnimationFrame(frame);
    const wheel = (e: WheelEvent) => {
      e.preventDefault();
      camera.current = Math.max(
        0,
        Math.min(
```

前段 app/battle.tsx:589 至 app/battle.tsx:638

```
          W - viewport.current,
          camera.current +
            (Math.abs(e.deltaX) > Math.abs(e.deltaY) ? e.deltaX : e.deltaY) *
              1.6,
        ),
      );
    };
    const el = canvas.current!;
    el.addEventListener('wheel', wheel, { passive: false });
    const visibility = () => {
      budget.reset(performance.now());
      accumulator = 0;
      keys.current.clear();
      if (document.hidden) audio.current?.setActive(false);
      if (document.hidden && game.current!.status === 'playing') {
        game.current!.status = 'paused';
        refresh();
      }
    };
    document.addEventListener('visibilitychange', visibility);
    return () => {
      stopped = true;
      unwatch();
      cancelAnimationFrame(raf);
      audio.current?.setActive(false);
      document.removeEventListener('visibilitychange', visibility);
      el.removeEventListener('wheel', wheel);
      if (messageTimer.current) clearTimeout(messageTimer.current);
    };
  }, [refresh, assetAttempt]);
  useEffect(() => {
    const pressedKeys = keys.current;
    const onKey = (e: KeyboardEvent) => {
      if (dialogueOpenRef.current) return;
      if (panel || e.altKey || e.ctrlKey || e.metaKey) return;
      const tag = (e.target as HTMLElement)?.tagName;
      if (
        tag === 'INPUT' ||
        tag === 'TEXTAREA' ||
        (e.target as HTMLElement)?.closest('[role="radiogroup"]')
      )
        return;
      const targetCard = game.current.players[0].hand.find(
        (h) => h.uid === selectedRef.current,
      );
      const targeting = targetCard ? needsTarget(targetCard.id) : false;
      if (e.key.toLowerCase() === 'a' || e.key.toLowerCase() === 'd') {
        keys.current.add(e.key.toLowerCase());
        return;
      }
```

前段 app/battle.tsx:639 至 app/battle.tsx:688

```
    if ((e.key === 'ArrowLeft' || e.key === 'ArrowRight') && !targeting) {
      e.preventDefault();
      moveCamera(camera.current + (e.key === 'ArrowRight' ? 85 : -85));
      return;
    }
    if (e.key === 'Escape') {
      setLogisticsUid(null);
      selectSquad(null);
      interruptCardHold();
      choose(null);
      hover.current = null;
      return;
    }
    if (e.code === 'Space' && tag !== 'BUTTON') {
      e.preventDefault();
      pause();
    }
    if (e.key.toLowerCase() === 'r') {
      e.preventDefault();
      drawCard();
      return;
    }
    if (/^[1-6]$/.test(e.key)) {
      const h = game.current!.players[0].hand[Number(e.key) - 1];
      if (h) choose(h.uid);
    }
    if ((e.key === 'ArrowLeft' || e.key === 'ArrowRight') && targeting) {
      e.preventDefault();
      pointerScreen.current = null;
      hover.current = Math.max(
        0,
        Math.min(
          W,
          (hover.current ?? 280) + (e.key === 'ArrowRight' ? 25 : -25),
        ),
      );
      if (hover.current > camera.current + viewport.current - 40)
        moveCamera(hover.current - viewport.current + 40);
      if (hover.current < camera.current + 40) moveCamera(hover.current - 40);
    }
    if (
      e.key === 'Enter' &&
      selectedRef.current !== null &&
      tag !== 'BUTTON'
    ) {
      e.preventDefault();
      const hand = game.current!.players[0].hand.find(
        (h) => h.uid === selectedRef.current,
      );
      if (hand)
```

前段 app/battle.tsx:689 至 app/battle.tsx:738

```
        execute(
          hand.uid,
          needsTarget(hand.id) ? (hover.current ?? 280) : undefined,
        );
      }
    };
    const onUp = (e: KeyboardEvent) => keys.current.delete(e.key.toLowerCase());
    window.addEventListener('keydown', onKey);
    window.addEventListener('keyup', onUp);
    return () => {
      window.removeEventListener('keydown', onKey);
      window.removeEventListener('keyup', onUp);
      pressedKeys.clear();
    };
  }, [
    panel,
    choose,
    pause,
    execute,
    moveCamera,
    drawCard,
    interruptCardHold,
    selectSquad,
  ]);
  useEffect(() => {
    const context = (
      document as unknown as {
        modelContext?: {
          registerTool: (
            tool: unknown,
            options: unknown,
          ) => Promise<void> | void;
        };
      }
    ).modelContext;
    if (!context?.registerTool) return;
    const lifecycle = new AbortController();
    const register = (tool: unknown) => {
      try {
        void Promise.resolve(
          context.registerTool(tool, { signal: lifecycle.signal }),
        ).catch(() => {});
      } catch {}
    };
    register({
      name: 'read_battle_state',
      description: '查看灰线当前战局、指挥点与手牌。',
      inputSchema: {
        type: 'object',
        properties: {},
```

前段 app/battle.tsx:739 至 app/battle.tsx:788

```
      additionalProperties: false,
    },
    annotations: { readOnlyHint: true },
    execute: () => snapshot(game.current!),
  });
  register({
    name: 'start_battle',
    description: '从整备状态开始一局灰线对战；不会重置进行中的战局。',
    inputSchema: {
      type: 'object',
      properties: {},
      additionalProperties: false,
    },
    annotations: { readOnlyHint: false },
    execute: () => {
      if (game.current!.status !== 'ready')
        return { ok: false, message: '当前不在整备状态' };
      start();
      return snapshot(game.current!);
    },
  });
  register({
    name: 'draw_battle_card',
    description:
      '消耗 2 点指挥点抽取 1 张卡，抽牌后冷却 1 秒。手牌已满、受干扰或资源不足时不消耗资源。',
    inputSchema: {
      type: 'object',
      properties: {},
      additionalProperties: false,
    },
    annotations: { readOnlyHint: false },
    execute: drawCard,
  });
  register({
    name: 'play_battle_card',
    description:
      '打出当前手牌。普通单位固定从己方基地入场，无需 x；需要落点的卡牌（烟幕、地雷、弹药空投、机降等）以 x 指定 0–3840 内的位置；其余技能无需 x。',
    inputSchema: {
      type: 'object',
      properties: { uid: { type: 'integer' }, x: { type: 'number' } },
      required: ['uid'],
      additionalProperties: false,
    },
    annotations: { readOnlyHint: false },
    execute: (input: unknown) => {
      const v = input as { uid?: unknown; x?: unknown };
      if (
        !v ||
        !Number.isInteger(v.uid) ||
        (v.x !== undefined &&
```

前段 app/battle.tsx:789 至 app/battle.tsx:838

```
            (typeof v.x !== 'number' || !Number.isFinite(v.x)))
        )
          return { ok: false, message: '无效的卡牌或坐标' };
        return execute(v.uid as number, v.x as number | undefined);
      },
    });
    return () => lifecycle.abort();
  }, [execute, start, drawCard]);
  const p = view.players[0],
    enemy = view.players[1],
    chosen = p.hand.find((h) => h.uid === selected),
    card = chosen ?? null;
  const inspectionCard = p.hand.find((h) => h.uid === inspectUid) ?? card;
  const active = view.status === 'playing';
  const units = view.units.filter(
      (u) => u.side === 0 && u.hp > 0 && !u.surrendered && !u.wounded,
    ),
    squads = new Set(units.map((u) => u.squad)).size;
  const deckCards = playerDeck.map((id) => CARDS[id]);
  const selectedMembers = units.filter(
    (u) => u.squad === selectedSquad && !u.rappelling,
  );
  const squadX =
    selectedMembers.reduce((n, u) => n + u.x, 0) /
    Math.max(1, selectedMembers.length);
  const squadY = selectedMembers.length
    ? CARDS[selectedMembers[0].id].air
      ? Math.max(...selectedMembers.map((u) => u.y)) + 140
      : Math.min(...selectedMembers.map((u) => unitSelectionBounds(u).y)) - 18
    : 0;
  const selectedLowUnit=selectedMembers.find(u=>logisticsAlert(u));
  const logisticsUnit=units.find(u=>u.uid===logisticsUid) ?? selectedLowUnit;
  const logisticsStatus=logisticsUnit&&logisticsAlert(logisticsUnit);
  const decisionAnchor=logisticsUid!==null ? logisticsAnchor : {x:squadX,y:squadY};
  const squadTrench = view.entrenchments.find((t) => t.squad === selectedSquad);
  const selectCard = (h: HandCard) => {
    didDrag.current = false;
    if (view.status === 'ready') {
      toast('点击「开始作战」后即可部署部队');
      return;
    }
    if (view.status === 'finished') return;
    choose(selected === h.uid ? null : h.uid);
    const lockout = playLockoutMessage(p.lockoutIn);
    if (lockout) toast(lockout);
  };
  const canvasX = (clientX: number) => {
    const rect = canvas.current!.getBoundingClientRect();
    return (
      ((clientX - rect.left) / rect.width) * viewport.current + camera.current
```

前段 app/battle.tsx:839 至 app/battle.tsx:888

```
    );
  };
  const handBounds = () => {
    if (!handArea.current) return null;
    const rect = handArea.current.getBoundingClientRect();
    const boxes = Array.from(
      handArea.current.querySelectorAll('.tactical-card'),
      (el) => el.getBoundingClientRect(),
    );
    if (!boxes.length) return null;
    return {
      left: Math.min(...boxes.map((b) => b.left)),
      right: Math.max(...boxes.map((b) => b.right)),
      top: rect.top,
      bottom: Math.max(rect.bottom, ...boxes.map((b) => b.bottom)),
    };
  };
  const changeAudio = (patch: Partial<AudioSettings>) => {
    const mixer = audio.current ?? getBattleAudio();
    mixer.configure(patch);
    setAudioSettings({ ...mixer.settings });
    if (mixer.settings.enabled) void mixer.unlock();
  };
  const toggleSound = () => changeAudio({ enabled: !sound });
  const changeOrder = useCallback((value: string) => {
    setOrder(game.current!, 0, value as Order);
    refresh();
  }, [refresh]);
  // The order controls only depend on the current order and match state.
  // Keep their provider/subscriptions out of periodic battle snapshots.
  const ordersBar = useMemo(() => (
    <div className="orders-bar">
      <span>
        <Flag size={14} /> 步兵指令
      </span>
      <RadioGroup
        className="orders-group"
        aria-label="步兵行动指令"
        value={p.order}
        onValueChange={changeOrder}
        disabled={view.status === 'finished'}
      >
        {(
          [
            ['hold', '驻守'],
            ['advance', '推进'],
            ['rush', '奔跑'],
            ['crouch', '蹲行'],
            ['prone', '卧倒'],
          ] as [Order, string][]
```

前段 app/battle.tsx:889 至 app/battle.tsx:938

```
        ).map(([value, label]) => (
          <label
            className={
              p.order === value ? 'order-option active' : 'order-option'
            }
            key={value}
          >
            <RadioGroupItem value={value} className="order-radio" />
            <span>{label}</span>
          </label>
        ))}
      </RadioGroup>
      <small>
        {p.order === 'rush'
          ? '行军速度 ×1.7'
          : p.order === 'crouch'
            ? '速度 ×0.55 · 受伤 −15%'
            : p.order === 'prone'
              ? '匍匐速度 ×0.25 · 受伤 −30%'
              : p.order === 'hold'
                ? '原地警戒，队员自主蹲伏还击'
                : '队员自主判断 · 交替掩护推进'}
      </small>
    </div>
  ), [p.order, view.status, changeOrder]);
  return (
    <main
      className={`game-shell ${touchMode ? 'touch-battle' : ''} ${view.status !== 'playing' || panel ? 'is-interrupted' : ''}`}
      data-art-phase={assetsReady ? 'ready' : assetProgress.stage}
      data-art-load-ms={assetLoadMs ?? undefined}
    >
      {missionId && (
        <CampaignDialogue
          missionId={missionId}
          open={dialogueOpen}
          onClose={closeDialogue}
          objectiveStatus={view.time > 0 ? missionStatus : undefined}
        />
      )}
      <div className="mobile-battle-hud">
        <div className="mobile-base mobile-base-own">
          <div>
            <span>我方指挥部</span>
            <strong>{Math.ceil(p.hp)}</strong>
          </div>
          <div
            className="mobile-hp"
            role="meter"
            aria-label="我方基地生命"
            aria-valuenow={p.hp}
```

前段 app/battle.tsx:939 至 app/battle.tsx:988

```
          aria-valuemin={0}
          aria-valuemax={MAX_HP}
        >
          <i style={{ width: `${p.hp / 10}%` }} />
        </div>
        <div className="mobile-command-points">
          <Zap size={14} />
          <strong>{Math.floor(p.energy)}</strong>
          <span>/ {p.energyCap} 指挥点</span>
          {(p.fogJammedUntil??0)>view.time&&<small>迷雾干扰 {Math.ceil(p.fogJammedUntil!-view.time)}秒</small>}
        </div>
        <small className="mobile-economy-readout">
          {(p.overdraftUntil??0)>view.time?'回点暂停':`+1 / ${p.energyInterval.toFixed(1)}秒`}
          {p.bondDueAt !== null
            ? ` · 预支 ${Math.max(0, Math.ceil(p.bondDueAt - view.time))}秒`
            : ''}
          {p.overdraftUntil !== null && p.overdraftUntil > view.time
            ? ` · 透支中 ${Math.max(0, Math.ceil(p.overdraftUntil - view.time))}秒`
            : ''}
          {p.suppressedUntil !== null && p.suppressedUntil > view.time
            ? ` · 遭压制 ${Math.max(0, Math.ceil(p.suppressedUntil - view.time))}秒`
            : ''}
        </small>
      </div>
      <button
        className="mobile-pause-control"
        aria-label="暂停作战"
        onClick={pause}
        disabled={!active}
      >
        <Pause size={17} />
        <span>{timeString(Math.ceil(timeLimit - view.time))}</span>
      </button>
      <div className="mobile-base mobile-base-enemy">
        <div>
          <span>敌方指挥部</span>
          <strong>{Math.ceil(enemy.hp)}</strong>
        </div>
        <div
          className="mobile-hp"
          role="meter"
          aria-label="敌方基地生命"
          aria-valuenow={enemy.hp}
          aria-valuemin={0}
          aria-valuemax={MAX_HP}
        >
          <i style={{ width: `${enemy.hp / 10}%` }} />
        </div>
        <small className="mobile-economy-readout">
          {DIFFICULTY_LABEL[difficulty]} · {DIFFICULTY_BONUS[difficulty]}
```

前段 app/battle.tsx:989 至 app/battle.tsx:1038

```
          </small>
        </div>
      </div>
      <div className="rotate-battle-hint">
        <RotateCcw size={34} />
        <strong>请横屏作战</strong>
        <span>转动手机，展开战场</span>
        <button onClick={onExit}>返回整备</button>
      </div>
      {mission &&
        view.campaign &&
        active &&
        !panel &&
        selectedMembers.length === 0 && (
          <button
            type="button"
            onClick={openDialogue}
            className="mission-status"
            aria-label={`查看任务简报：${mission.goal}`}
          >
            <strong>{mission.title}</strong>
            <span>{missionStatus}</span>
          </button>
        )}
      <header className="masthead">
        <div className="brand">
          <span className="brand-symbol">
            <span />
            <span />
            <span />
          </span>
          <div className="wordmark">
            灰线<span>GREYLINE</span>
          </div>
          <span className="brand-meta">
            战术卡牌
            <br />
            TACTICAL CARD WARFARE
          </span>
        </div>
        <div className="header-actions">
          <button className="text-button return-home" onClick={onExit}>
            返回整备
          </button>
          <span className="live-label">
            <i /> {DIFFICULTY_LABEL[difficulty]} ·{' '}
            {DIFFICULTY_BONUS[difficulty]}
          </span>
          <button
            className="text-button"
```

前段 app/battle.tsx:1039 至 app/battle.tsx:1088

```
          aria-label="作战手册"
          onClick={() => openPanel('guide')}
        >
          <BookOpen size={16} />
          作战手册
        </button>
        <button
          className="icon-button"
          aria-label={sound ? '关闭声音' : '开启声音'}
          title={sound ? '关闭声音' : '开启声音'}
          onClick={toggleSound}
        >
          {sound ? <Volume2 size={18} /> : <VolumeX size={18} />}
        </button>
        <button
          className="icon-button"
          aria-label={view.status === 'paused' ? '继续作战' : '暂停作战'}
          disabled={view.status === 'ready' || view.status === 'finished'}
          onClick={pause}
        >
          {view.status === 'paused' ? (
            <Play size={18} />
          ) : (
            <Pause size={18} />
          )}
        </button>
      </div>
    </header>
    <section className="match-bar" aria-label="双方基地状态">
      <div className="team team-blue">
        <div className="team-insignia">
          <Shield size={23} />
        </div>
        <div className="team-data">
          <div className="team-heading">
            <strong>蓝方指挥部</strong>
            <span>YOU</span>
            <b>
              {Math.ceil(p.hp).toLocaleString()} <small>/ 1,000</small>
            </b>
          </div>
          <div
            className="hp-track"
            role="meter"
            aria-label="蓝方基地生命"
            aria-valuenow={p.hp}
            aria-valuemin={0}
            aria-valuemax={MAX_HP}
          >
            <i style={{ width: `${p.hp / 10}%` }} />
```

前段 app/battle.tsx:1089 至 app/battle.tsx:1138

```
          </div>
        </div>
      </div>
      <div className="match-clock">
        <span>
          {view.status === 'ready'
            ? '等待指令'
            : view.status === 'paused'
              ? '战场暂停'
              : view.status === 'finished'
                ? '作战结束'
                : (mission?.title ?? '作战进行中')}
        </span>
        <strong>{timeString(Math.ceil(timeLimit - view.time))}</strong>
        {mission && view.campaign ? (
          <button
            type="button"
            className="mission-clock-goal"
            onClick={openDialogue}
            aria-label="查看任务简报"
          >
            {missionStatus}
          </button>
        ) : (
          <div className="clock-dots">
            <i />
            <i />
            <i />
          </div>
        )}
      </div>
      <div className="team team-red">
        <div className="team-data">
          <div className="team-heading">
            <b>
              {Math.ceil(enemy.hp).toLocaleString()} <small>/ 1,000</small>
            </b>
            <span>AI</span>
            <strong>红方指挥部</strong>
          </div>
          <div
            className="hp-track"
            role="meter"
            aria-label="红方基地生命"
            aria-valuenow={enemy.hp}
            aria-valuemin={0}
            aria-valuemax={MAX_HP}
          >
            <i style={{ width: `${enemy.hp / 10}%` }} />
          </div>
```

前段 app/battle.tsx:1139 至 app/battle.tsx:1188

```
          </div>
          <div className="team-insignia">
            <Flag size={23} />
          </div>
        </div>
      </section>
      <section
        className={`battlefield ${card?.targetGround ? 'is-targeting' : ''}`}
        aria-label="像素战场"
      >
        <canvas
          ref={canvas}
          width={viewportWidth}
          height={H}
          tabIndex={0}
          aria-label="左右拖动战场移动视野。拖出底部手牌区并松手出牌，单位从己方基地入场。数字键选牌，回车出牌；目标技能可用方向键调整位置。"
          onPointerDown={(e) => {
            if (e.button !== 0 || cardGesture.current) return;
            if (e.pointerType !== 'mouse') setTouchMode(true);
            if (!e.isPrimary || dragView.current) {
              didDrag.current = true;
              return;
            }
            dragView.current = {
              x: e.clientX,
              y: e.clientY,
              camera: camera.current,
              id: e.pointerId,
            };
            didDrag.current = false;
            e.currentTarget.setPointerCapture(e.pointerId);
          }}
          onPointerMove={(e) => {
            const gesture = dragView.current;
            if (gesture) {
              if (gesture.id !== e.pointerId) return;
              const dx = e.clientX - gesture.x,
                dy = e.clientY - gesture.y;
              if (Math.abs(dx) > 8 && Math.abs(dx) > Math.abs(dy))
                didDrag.current = true;
              if (didDrag.current) {
                const rect = e.currentTarget.getBoundingClientRect();
                moveCamera(
                  gesture.camera - (dx / rect.width) * viewport.current,
                );
                pointerScreen.current = null;
                return;
              }
            }
            if (e.pointerType === 'mouse') {
```

前段 app/battle.tsx:1189 至 app/battle.tsx:1238

```
            hover.current = canvasX(e.clientX);
            pointerScreen.current = hover.current - camera.current;
          }
        }}
        onPointerUp={(e) => {
          const gesture = dragView.current;
          if (!gesture || gesture.id !== e.pointerId) return;
          dragView.current = null;
          if (
            !didDrag.current &&
            selectedRef.current === null &&
            game.current.status === 'playing'
          ) {
            const rect = e.currentTarget.getBoundingClientRect();
            const warning=pickLogisticsIndicator(game.current,canvasX(e.clientX),
              ((e.clientY-rect.top)/rect.height)*H,e.pointerType!=='mouse');
            if(warning){
              openLogistics(warning.uid);
              if(e.currentTarget.hasPointerCapture(e.pointerId))e.currentTarget.releasePointerCapture(e.pointerId);
              return;
            }
            setLogisticsUid(null);
            const found = pickSquad(
              game.current,
              0,
              canvasX(e.clientX),
              ((e.clientY - rect.top) / rect.height) * H,
              e.pointerType !== 'mouse',
            );
            if (found !== null && found !== selectedSquadRef.current) {
              const result = selectUnitGroup(game.current, 0, found);
              if (!result.ok) toast(result.message);
              refresh();
            }
            selectSquad(found === selectedSquadRef.current ? null : found);
          }
          if (e.currentTarget.hasPointerCapture(e.pointerId))
            e.currentTarget.releasePointerCapture(e.pointerId);
        }}
        onPointerCancel={(e) => {
          if (dragView.current?.id === e.pointerId) {
            dragView.current = null;
            didDrag.current = true;
          }
        }}
        onLostPointerCapture={(e) => {
          if (dragView.current?.id === e.pointerId) {
            dragView.current = null;
            didDrag.current = true;
          }
```

前段 app/battle.tsx:1239 至 app/battle.tsx:1288

```
        }}
        onPointerLeave={() => {
          if (!touchMode) {
            hover.current = null;
            pointerScreen.current = null;
          }
        }}
      />
      {active&&!panel&&logisticsUnit&&logisticsStatus&&
        decisionAnchor.x>=cameraView&&decisionAnchor.x<=cameraView+viewportWidth&&(
        <UnitLogisticsFan x={(decisionAnchor.x-cameraView)/viewportWidth*100}
          y={decisionAnchor.y/H*100} unit={logisticsUnit} reason={logisticsStatus.reason}
          radar={CARDS[logisticsUnit.id].airRadarRange?{on:!logisticsUnit.radarOff,onToggle:()=>{toggleRadar(game.current,logisticsUnit.uid);refresh();}}:undefined}
          onClose={()=>{setLogisticsUid(null);selectSquad(null);}} onOrder={order=>{
            if(issueLogisticsOrder(game.current,logisticsUnit.uid,order)){
              const immobile=(CARDS[logisticsUnit.id].static&&!CARDS[logisticsUnit.id].emplacement)||vehicleOutOfFuel(logisticsUnit)||isImmobilized(logisticsUnit);
              toast(order==='advance'?(immobile?'前进指令已下达，需先补油或修复履带':'继续前进指令已下达'):
                order==='hold'?'原地待命，保持警戒':immobile?'无法移动，原地等待补给':'正在前往补给点');
              setLogisticsUid(null);selectSquad(null);refresh();
            }
          }}/>
      )}
      {active &&
        !panel &&
        !logisticsStatus &&
        selectedMembers.length > 0 &&
        squadX >= cameraView &&
        squadX <= cameraView + viewportWidth && (
          <SquadMenu
            x={(((squadMenuAnchor?.x ?? squadX) - cameraView) / viewportWidth) * 100}
            y={((squadMenuAnchor?.y ?? squadY) / H) * 100}
            name={CARDS[selectedMembers[0].id].name}
            count={selectedMembers.length}
            unitLabel={
              CARDS[selectedMembers[0].id].members
                ? '人'
                : CARDS[selectedMembers[0].id].air
                  ? '架'
                  : CARDS[selectedMembers[0].id].emplacement
                    ? '门'
                    : '辆'
            }
            radar={CARDS[selectedMembers[0].id].airRadarRange?{on:!selectedMembers[0].radarOff,onToggle:()=>{toggleRadar(game.current,selectedMembers[0].uid);refresh();}}:undefined}
            orders={ordersForUnit(selectedMembers[0].id)}
            order={
              selectedMembers[0].squadOrder ??
              (selectedMembers[0].escortTankUid !== undefined
                || automaticFireSupport(selectedMembers[0].id)
                ? 'escort'
                : undefined)
```

前段 app/battle.tsx:1289 至 app/battle.tsx:1338

```
            }
            progress={squadTrench?.progress}
            constructionLabel={trenchConstructionLabel(selectedMembers, squadTrench, view.time)}
            onOrder={(order) => {
              const result = setSquadOrder(
                game.current,
                0,
                selectedSquad!,
                order,
              );
              if (!result.ok) toast(result.message);
              else if (order === 'hold') toast('修筑阵地指令已下达');
              refresh();
            }}
            onClose={() => selectSquad(null)}
          />
        )}
        <div className="field-top">
          <div className="location">
            <Crosshair size={15} />
            <span>
              07 <b>无名村落</b>
            </span>
            <small>48° N · 23° E</small>
          </div>
          <span className="weather">
            林间前线 <span> / </span> 多云
          </span>
        </div>
        <div className="field-ruler">
          <span>
            {touchMode ? '← 左右滑动战场 →' : '← 拖动 / A、D 移动视野 →'}
          </span>
          <i />
          <span>
            {Math.round(cameraView)} — {Math.round(cameraView + viewportWidth)}{' '}
            / {W}
          </span>
        </div>
        {active && (
          <div className="field-status">
            <span className="live-dot" />{' '}
            {draggedCard
              ? draggedCard.outside
                ? '松手使用 · 拖回手牌区取消'
                : '拖出底部手牌区使用'
              : '拖出手牌并松手下令 · 长按查看详情'}
            {card && (
              <button onClick={() => choose(null)} aria-label="取消选择">
                <X size={13} />
```

前段 app/battle.tsx:1339 至 app/battle.tsx:1388

```
            取消
          </button>
        )}
      </div>
    )}
    {p.morale > 0 && (
      <div className="buff-label">
        <Sparkles size={13} />
        士气鼓舞 · {Math.ceil(p.morale)}s
      </div>
    )}
    {p.recon > 0 && (
      <div className="recon-label">
        <Binoculars size={14} /> 校射 +20% · {Math.ceil(p.recon)}s
      </div>
    )}
    {view.status === 'ready' && !dialogueOpen && (
      <div className="start-scrim">
        <div className="launch">
          <div className="operation">
            <i /> OPERATION: GREYLINE <i />
          </div>
          <h1>战线，由你推进。</h1>
          <p>
            {mission
              ? mission.goal
              : '部署部队，下达指令。夺下村落另一端的指挥部。'}
          </p>
          <button
            className="primary-button"
            data-art-phase={assetProgress.stage}
            data-art-completed={assetProgress.completed}
            data-art-total={assetProgress.total}
            data-art-load-ms={assetLoadMs ?? undefined}
            onClick={() => {
              if (assetError) setAssetAttempt(attempt => attempt + 1);
              else start();
            }}
            disabled={!assetsReady && !assetError}
          >
            {assetError
              ? '重新加载素材'
              : assetsReady
                ? '开始作战'
                : assetProgress.stage === 'download' ? '正在加载战场…' : '正在部署装备…'}
            <ArrowRight size={19} />
          </button>
          <div className="launch-meta">
            <span>1 VS 1</span>
            <i />
```

后段 minigame/src/ui/lobby.ts:128 至 minigame/src/ui/lobby.ts:185

```
      const idx = MAP_IDS.indexOf(lobbyState.mapId);
      const next = MAP_IDS[(idx + 1) % MAP_IDS.length];
      lobbyState.setMap(next);
      this.refreshSidebarLabels();
    };
    this.addChild(this.mapBtn);
    y += 50;
    this.nightBtn = new Button('', SIDEBAR_W - 16, 40, { fontSize: 11 });
    this.nightBtn.x = 8;
    this.nightBtn.y = y;
    this.nightBtn.onTap = () => {
      lobbyState.setNight(!lobbyState.night);
      this.refreshSidebarLabels();
    };
    this.addChild(this.nightBtn);
    y += 46;
    this.diffBtn = new Button('', SIDEBAR_W - 16, 40, { fontSize: 11 });
    this.diffBtn.x = 8;
    this.diffBtn.y = y;
    this.diffBtn.onTap = () => this.setSubpage('settings');
    this.addChild(this.diffBtn);
    this.refreshSidebarLabels();
  }
  private refreshSidebarLabels(): void {
    const map = MAPS[lobbyState.mapId];
    this.mapBtn.label = `地图: ${map.name}`;
    this.nightBtn.label = lobbyState.night ? '时段: 夜间' : '时段: 昼间';
    this.diffBtn.label = `对手: ${DIFFICULTY_LABEL[lobbyState.difficulty]}`;
  }
  // ── Content pages ──────────────────────────────────────────────
  private setSubpage(page: LobbyPage): void {
    this.subpage = page;
    this.scrollY = 0;
    this.buildContent();
    // Map/night selectors hidden on campaign (mission defines its own map)
    this.mapBtn.visible = page !== 'campaign';
    this.nightBtn.visible = page !== 'campaign';
  }
  private clearContent(): void {
    const keep = new Set<Widget>([
      ...this.navButtons,
      this.mapBtn,
      this.nightBtn,
      this.diffBtn,
    ]);
    for (const c of [...this.children]) {
      if (!keep.has(c)) this.removeChild(c);
    }
  }
  private buildContent(): void {
```

后段 minigame/src/ui/lobby.ts:186 至 minigame/src/ui/lobby.ts:238

```
    this.clearContent();
    if (this.subpage === 'home') this.buildHome();
    else if (this.subpage === 'settings') this.buildSettings();
    else if (this.subpage === 'campaign') this.buildCampaign();
    // guide is pure text, drawn in drawSelf
  }
  private buildHome(): void {
    this.startBtn = new Button('开始游戏', 220, 56, {
      fontSize: 20,
      bold: true,
      bg: COLORS.accent,
      bgPressed: COLORS.accentDark,
      textColor: '#1a1f1a',
      border: COLORS.accentDark,
      radius: 6,
    });
    this.startBtn.x = this.contentX() + (this.contentW() - 220) / 2;
    this.startBtn.y = this.screenH - 96;
    this.startBtn.onTap = () => {
      const err = lobbyState.begin();
      if (err) this.toast(err);
    };
    this.addChild(this.startBtn);
  }
  private buildSettings(): void {
    const x = this.contentX() + 24;
    let y = 24;
    // Difficulty buttons
    const diffs: Difficulty[] = ['standard', 'veteran', 'elite'];
    const bw = 150;
    for (const d of diffs) {
      const btn = new Button(
        `${DIFFICULTY_LABEL[d]}\n${DIFFICULTY_BONUS[d]}`,
        bw,
        52,
        {
          fontSize: 12,
          bg:
            lobbyState.difficulty === d ? COLORS.green : COLORS.bgPanelLight,
          border:
            lobbyState.difficulty === d ? COLORS.greenLight : COLORS.border,
        },
      );
      btn.x = x + diffs.indexOf(d) * (bw + 10);
      btn.y = y;
      const dd = d;
      btn.onTap = () => {
        lobbyState.setDifficulty(dd);
        this.buildContent();
        this.refreshSidebarLabels();
```

后段 minigame/src/ui/lobby.ts:239 至 minigame/src/ui/lobby.ts:292

```
    };
    this.addChild(btn);
  }
  y += 66;
  // Sound toggle
  const audio = getBattleAudio().settings;
  const toggle = new Toggle(44, 22, audio.enabled);
  toggle.x = x;
  toggle.y = y;
  toggle.onChange = (v) => {
    getBattleAudio().configure({ enabled: v });
  };
  this.addChild(toggle);
  y += 36;
  // Music volume
  const musicSlider = new Slider(260, 24, audio.music);
  musicSlider.x = x;
  musicSlider.y = y;
  musicSlider.onChange = (v) => {
    getBattleAudio().configure({ music: v });
  };
  this.addChild(musicSlider);
  y += 36;
  // SFX volume
  const sfxSlider = new Slider(260, 24, audio.effects);
  sfxSlider.x = x;
  sfxSlider.y = y;
  sfxSlider.onChange = (v) => {
    getBattleAudio().configure({ effects: v });
  };
  this.addChild(sfxSlider);
}
private buildCampaign(): void {
  const x = this.contentX() + 16;
  let y = 16;
  for (let i = 0; i < MISSIONS.length; i++) {
    const m = MISSIONS[i];
    const done = lobbyState.completed.includes(m.id);
    const unlocked = lobbyState.isMissionUnlocked(i);
    const btn = new Button(
      `${m.chapter} ${m.title}  ${done ? '✓' : unlocked ? '' : '□'}`,
      150,
      40,
      {
        fontSize: 11,
        bg:
          this.selectedMission === m.id
            ? COLORS.green
            : COLORS.bgPanel,
      },
```

后段 minigame/src/ui/lobby.ts:293 至 minigame/src/ui/lobby.ts:346

```
      );
      btn.x = x;
      btn.y = y;
      btn.enabled = unlocked;
      btn.onTap = () => {
        this.selectedMission = m.id;
        this.buildContent();
      };
      this.addChild(btn);
      y += 46;
    }
    // Start button
    const idx = MISSIONS.findIndex((m) => m.id === this.selectedMission);
    const unlocked = lobbyState.isMissionUnlocked(idx);
    const startBtn = new Button(
      unlocked ? '进入战役 →' : '完成上一章后解锁',
      170,
      40,
      {
        fontSize: 13,
        bold: true,
        bg: unlocked ? COLORS.accent : COLORS.bgPanelLight,
        textColor: unlocked ? '#1a1f1a' : COLORS.textMuted,
      },
    );
    startBtn.x = x;
    startBtn.y = y + 8;
    startBtn.enabled = unlocked;
    startBtn.onTap = () => {
      const err = lobbyState.begin(undefined, this.selectedMission);
      if (err) this.toast(err);
    };
    this.addChild(startBtn);
  }
  // ── Images ──────────────────────────────────────────────────────────
  private loadHomeBg(): void {
    const img = new Image();
    img.onload = () => {
      this.homeBg = img;
    };
    img.src = assetUrl('/art/home-camp-v14.png');
  }
  private loadMissionBg(url: string): void {
    if (this.missionBgUrl === url) return;
    this.missionBgUrl = url;
    this.missionBg = null;
    const img = new Image();
    img.onload = () => {
      if (this.missionBgUrl === url) this.missionBg = img;
    };
```

后段 minigame/src/ui/lobby.ts:347 至 minigame/src/ui/lobby.ts:403

```
    img.src = assetUrl(url);
  }
  // ── Toast ──────────────────────────────────────────────────────────
  private toast(msg: string): void {
    this.toastMsg = msg;
    this.toastTime = Date.now();
  }
  // ── Lifecycle ──────────────────────────────────────────────────────
  onEnter(): void {
    this.refreshSidebarLabels();
    this.buildContent();
    const mixer = getBattleAudio();
    mixer.prefetch();
    mixer.setActive(true);
  }
  handleTouch(ev: TouchEvent): boolean {
    if (!this.audioUnlocked && ev.type === 'down') {
      this.audioUnlocked = true;
      void getBattleAudio().unlock();
      getBattleAudio().setActive(true);
    }
    if (super.handleTouch(ev)) return true;
    // Content scrolling for guide / campaign text
    if (this.subpage !== 'guide' && this.subpage !== 'campaign')
      return false;
    const p = ev.changed[0];
    if (!p || p.x < this.contentX()) return false;
    if (ev.type === 'down') {
      this.scrollTouchId = p.id;
      this.scrollStartY = p.y;
      this.scrollStartScroll = this.scrollY;
      return true;
    }
    if (ev.type === 'move' && this.scrollTouchId === p.id) {
      this.scrollY = clamp(
        this.scrollStartScroll - (p.y - this.scrollStartY),
        0,
        this.scrollMax,
      );
      return true;
    }
    if (ev.type === 'up' && this.scrollTouchId === p.id) {
      this.scrollTouchId = -1;
      return true;
    }
    return false;
  }
  // ── Drawing ────────────────────────────────────────────────────────
  protected drawSelf(ctx: CanvasRenderingContext2D): void {
    // Background
```

后段 minigame/src/ui/lobby.ts:404 至 minigame/src/ui/lobby.ts:460

```
    ctx.fillStyle = COLORS.bg;
    ctx.fillRect(0, 0, this.screenW, this.screenH);
    this.drawSidebar(ctx);
    const cx = this.contentX();
    ctx.save();
    ctx.beginPath();
    ctx.rect(cx, 0, this.contentW(), this.screenH);
    ctx.clip();
    ctx.translate(0, -this.scrollY);
    if (this.subpage === 'home') this.drawHome(ctx);
    else if (this.subpage === 'guide') this.drawGuide(ctx);
    else if (this.subpage === 'campaign') this.drawCampaignBrief(ctx);
    // settings widgets draw themselves; draw labels here
    if (this.subpage === 'settings') this.drawSettingsLabels(ctx);
    ctx.restore();
    // Toast
    if (this.toastMsg && Date.now() - this.toastTime < 3000) {
      const alpha = Math.min(1, (3000 - (Date.now() - this.toastTime)) / 500);
      ctx.globalAlpha = alpha;
      const w = measureText(ctx, this.toastMsg, 13) + 32;
      const x = this.screenW / 2 - w / 2;
      const y = this.screenH - 60;
      drawPanel(ctx, x, y, w, 32, COLORS.overlay, COLORS.accent);
      drawTextCentered(ctx, this.toastMsg, this.screenW / 2, y + 16, 13);
      ctx.globalAlpha = 1;
    }
  }
  private drawSidebar(ctx: CanvasRenderingContext2D): void {
    drawPanel(ctx, 0, 0, SIDEBAR_W, this.screenH, COLORS.bgPanel, COLORS.border);
    // Brand
    drawTextLeft(ctx, '灰线', 12, 20, 20, COLORS.accent, 'bold');
    drawTextLeft(ctx, 'GREYLINE · v181', 12, 40, 9, COLORS.textDim);
    // Gold + deck status at bottom
    const gold = lobbyState.collection.gold;
    drawTextLeft(ctx, `金币 ${gold}`, 12, this.screenH - 44, 12, COLORS.gold);
    drawTextLeft(
      ctx,
      `当前编队 ${lobbyState.deck.length}/25`,
      12,
      this.screenH - 24,
      12,
      lobbyState.deck.length === 20 ? COLORS.greenLight : COLORS.redLight,
    );
  }
  private drawHome(ctx: CanvasRenderingContext2D): void {
    const cx = this.contentX();
    const cw = this.contentW();
    if (this.homeBg) {
      // Cover fit
      const imgRatio = this.homeBg.width / this.homeBg.height;
```

后段 minigame/src/ui/lobby.ts:461 至 minigame/src/ui/lobby.ts:511

```
    const boxRatio = cw / this.screenH;
    let dw = cw;
    let dh = this.screenH;
    let dx = cx;
    let dy = 0;
    if (imgRatio > boxRatio) {
      dw = this.screenH * imgRatio;
      dx = cx - (dw - cw) / 2;
    } else {
      dh = cw / imgRatio;
      dy = -(dh - this.screenH) / 2;
    }
    ctx.drawImage(this.homeBg, dx, dy, dw, dh);
  }
  // Title
  drawTextCentered(
    ctx,
    '灰线 · 现代战争像素对战',
    cx + cw / 2,
    48,
    22,
    COLORS.text,
    'bold',
  );
 }
 private drawGuide(ctx: CanvasRenderingContext2D): void {
  const cx = this.contentX() + 20;
  const cw = this.contentW() - 40;
  let y = 20;
  drawTextLeft(ctx, '作战手册', cx, y, 20, COLORS.accent, 'bold');
  y += 36;
  const sections: [string, string][] = [
    ['架空设定', FICTIONAL_SETTING_NOTICE],
    [
      '胜利条件',
      '摧毁敌方指挥部。十分钟后仍未分出胜负，则比较双方基地剩余生命。',
    ],
    [
      '出牌与抽牌',
      '把卡牌拖出底部扇形区域后松手，即可使用；拖回则取消。长按查看详情。单位从己方基地出发，烟幕、地雷和机降落点使用松手位置。机降直升机会飞抵落点后放下步兵，途中可被防空击落。\n点击牌堆，消耗 2 点指挥点抽牌，冷却 9 秒。手牌最多 6 张。\n双方开局 2 点，每 3.6 秒恢复 1 点。后勤可加快回点，扩编可增加上限；难度设置会注明 AI 的额外回点速度。',
    ],
    [
      '查看战场',
      '左右拖动画面移动视野。手机横屏游玩。',
    ],
    [
      '机步协同',
      '步兵会自动伴随附近友军坦克，保持后方距离。点击小队可选择伴随、据守、撤退、进攻或警戒；手动进攻会解除伴随。缺少有效反甲或防空保护时，步兵会持续撤离重装火力范围。',
    ],
    [
```

后段 minigame/src/ui/lobby.ts:512 至 minigame/src/ui/lobby.ts:565

```
      '组建编队',
      '在卡组中选择 25 张牌，每种卡各有数量上限。带上反坦克与防空兵种；AI 使用自己的卡组，双方各抽各的。',
    ],
  ];
  for (const [title, body] of sections) {
    drawTextLeft(ctx, title, cx, y, 14, COLORS.greenLight, 'bold');
    y += 22;
    y = drawWrapped(ctx, body, cx, y, cw, 12, COLORS.text, 'normal', 19) + 12;
  }
  this.scrollMax = Math.max(0, y - this.screenH);
}
private drawCampaignBrief(ctx: CanvasRenderingContext2D): void {
  const idx = MISSIONS.findIndex((m) => m.id === this.selectedMission);
  const m = MISSIONS[idx];
  const cx = this.contentX() + 180;
  const cw = this.contentW() - 200;
  let y = 16;
  drawTextLeft(
    ctx,
    '故事战役 · 六章 — 断线之地',
    cx,
    y,
    16,
    COLORS.accent,
    'bold',
  );
  y += 28;
  // Landscape image
  this.loadMissionBg(MAPS[m.mapId].backgroundAsset);
  const imgH = Math.min(110, this.screenH * 0.3);
  if (this.missionBg) {
    ctx.drawImage(this.missionBg, cx, y, cw, imgH);
  } else {
    drawPanel(ctx, cx, y, cw, imgH, COLORS.bgPanelLight, COLORS.border);
  }
  drawTextLeft(ctx, m.region, cx + 8, y + 14, 11, COLORS.text, 'bold');
  drawTextLeft(ctx, m.title, cx + 8, y + 30, 15, COLORS.text, 'bold');
  y += imgH + 12;
  y = drawWrapped(ctx, m.briefing, cx, y, cw, 12, COLORS.text) + 10;
  drawTextLeft(ctx, `任务: ${m.goal}`, cx, y, 12, COLORS.greenLight, 'bold');
  y += 20;
  y =
    drawWrapped(ctx, `已知部署: ${m.preparation}`, cx, y, cw, 11, COLORS.textDim) +
    10;
  drawTextLeft(
    ctx,
    `${DIFFICULTY_LABEL[lobbyState.difficulty]} · ${DIFFICULTY_BONUS[lobbyState.difficulty]}`,
    cx,
    y,
    11,
```

后段 minigame/src/ui/lobby.ts:566 至 minigame/src/ui/lobby.ts:616

```
      COLORS.textDim,
    );
    y += 18;
    drawTextLeft(
      ctx,
      '使用当前25张编队 · 战役进度保存在本机 · 进场先听无线电简报',
      cx,
      y,
      10,
      COLORS.textMuted,
    );
    y += 16;
    this.scrollMax = Math.max(0, y - this.screenH);
  }
  private drawSettingsLabels(ctx: CanvasRenderingContext2D): void {
    const x = this.contentX() + 24;
    drawTextLeft(ctx, '对手难度', x, 14, 14, COLORS.accent, 'bold');
    drawTextLeft(
      ctx,
      '双方开局均为 2 点指挥点。资源优势只影响 AI 回点速度。',
      x,
      100,
      11,
      COLORS.textDim,
    );
    drawTextLeft(ctx, '游戏声音', x, 128, 13, COLORS.text);
    const audio = getBattleAudio().settings;
    drawTextLeft(
      ctx,
      `音乐音量 ${Math.round(audio.music * 100)}%`,
      x + 320,
      152,
      12,
      COLORS.text,
    );
    drawTextLeft(
      ctx,
      `音效音量 ${Math.round(audio.effects * 100)}%`,
      x + 320,
      188,
      12,
      COLORS.text,
    );
    drawTextLeft(
      ctx,
      '自动保存，进入战场后同样生效。',
      x,
      224,
      11,
      COLORS.textDim,
```

后段 minigame/src/ui/lobby.ts:617 至 minigame/src/ui/shop.ts:48

```
    );
  }
}
/**
 * Shop screen: buy card packs, watch the tear animation, reveal draws
 * (fan for single packs, scrollable grid for ten-packs), watch ads for
 * gold, and browse the owned-card collection.
 * Replaces app/shop.tsx.
 */
import {
  Screen,
  Button,
  ScrollList,
  Widget,
  type Router,
} from './framework';
import {
  COLORS,
  clamp,
  drawTextCentered,
  drawTextLeft,
  drawTextRight,
  roundRect,
} from './theme';
import { drawCardFace, drawCardMini, loadCardImage } from './card-render';
import { CardDetailDialog } from './card-detail-dialog';
import { lobbyState } from '../lobby-state';
import { CARDS, type CardId } from '@/game/engine';
import {
  PACK_COST,
  TEN_PACK_COST,
  AD_REWARD,
  AD_COOLDOWN_MS,
  RARITY_LABEL,
  RARITY_RATE,
  collectionProgress,
  grantAdReward,
  openPack,
  openTenPacks,
  ownedCount,
  rarityOf,
  type PackDraw,
  type PackResult,
  type Rarity,
} from '@/game/collection';
import {
  PACK_CANVAS,
  PACK_DURATION,
  packFrameRect,
  packMotion,
```

后段 minigame/src/ui/shop.ts:49 至 minigame/src/ui/shop.ts:108

```
} from '@/game/pack-animation';
import { assetUrl } from '@/game/asset-url';
const TOP_H = 40;
const BOTTOM_H = 100;
const LEFT_W = 216;
const ROW_H = 50;
const RARITY_COLOR: Record<Rarity, string> = {
  common: '#8b9578',
  rare: '#c0c8d0',
  epic: '#e8c25a',
  legendary: '#b06fd0',
};
const AD_COOLDOWN_STORAGE = 'greyline-ad-cooldown';
// ── Owned-card list row ──────────────────────────────────────────────
class OwnedRow extends Widget {
  id: CardId;
  count: number;
  constructor(id: CardId, count: number) {
    super();
    this.id = id;
    this.count = count;
  }
  protected drawSelf(ctx: CanvasRenderingContext2D): void {
    const c = CARDS[this.id];
    drawCardMini(ctx, this.id, 6, 4, 28);
    drawTextLeft(ctx, c.name, 44, 13, 12, COLORS.text);
    drawTextLeft(
      ctx,
      RARITY_LABEL[rarityOf(this.id)],
      44,
      30,
      9,
      RARITY_COLOR[rarityOf(this.id)],
    );
    drawTextRight(ctx, `×${this.count}`, this.w - 8, 26, 12, COLORS.textDim);
  }
}
// ── Pack tear animation (full screen) ────────────────────────────────
class TearWidget extends Widget {
  onDone: (() => void) | null = null;
  private atlas: HTMLImageElement | null = null;
  private t = 0;
  private finished = false;
  private loadTimer: ReturnType<typeof setTimeout> | null = null;
  constructor(w: number, h: number) {
    super();
    this.w = w;
    this.h = h;
    this.loadTimer = setTimeout(() => this.finish(), 5000);
    loadCardImage(assetUrl('art/pack-tear-v139.png'))
```

后段 minigame/src/ui/shop.ts:109 至 minigame/src/ui/shop.ts:161

```
      .then((img) => {
        if (this.loadTimer) clearTimeout(this.loadTimer);
        this.loadTimer = null;
        this.atlas = img;
      })
      .catch(() => this.finish());
  }
  update(dt: number): void {
    if (!this.atlas || this.finished) return;
    this.t += dt;
    if (this.t >= PACK_DURATION) this.finish();
  }
  private finish(): void {
    if (this.finished) return;
    this.finished = true;
    if (this.loadTimer) {
      clearTimeout(this.loadTimer);
      this.loadTimer = null;
    }
    this.onDone?.();
  }
  protected drawSelf(ctx: CanvasRenderingContext2D): void {
    ctx.fillStyle = COLORS.bg;
    ctx.fillRect(0, 0, this.w, this.h);
    if (!this.atlas) {
      drawTextCentered(ctx, '准备拆封…', this.w / 2, this.h / 2, 14, COLORS.textDim);
      return;
    }
    const motion = packMotion(this.t);
    const s = Math.min(this.w / PACK_CANVAS.width, this.h / PACK_CANVAS.height) * 0.85;
    const img = this.atlas;
    const cell = packFrameRect(img.width, img.height, motion.frame);
    ctx.save();
    ctx.imageSmoothingEnabled = false;
    ctx.globalAlpha = motion.alpha;
    ctx.translate(this.w / 2, this.h / 2);
    ctx.scale(s, s);
    ctx.translate(motion.x - PACK_CANVAS.width / 2, motion.y - PACK_CANVAS.height / 2);
    ctx.drawImage(
      img,
      cell.x,
      cell.y,
      cell.width,
      cell.height,
      0,
      0,
      cell.width,
      cell.height,
    );
    ctx.restore();
```

后段 minigame/src/ui/shop.ts:162 至 minigame/src/ui/shop.ts:218

```
    drawTextCentered(ctx, '点击跳过', this.w / 2, this.h - 24, 11, COLORS.textMuted);
  }
  onTouchDown(): boolean {
    return true;
  }
  onTouchUp(): void {
    this.finish();
  }
}
// ── Reveal area (fan for 5, scrollable grid for 50) ──────────────────────
interface RevealCallbacks {
  onFlip: () => void;
  onFlipAll: () => void;
  onDetail: (id: CardId) => void;
}
class RevealAreaWidget extends Widget {
  private draws: PackDraw[];
  private flipped: boolean[];
  private prevOwned: Partial<Record<CardId, number>>;
  private cb: RevealCallbacks;
  private backImg: HTMLImageElement | null = null;
  private ten: boolean;
  // Grid scroll state
  private scrollY = 0;
  private maxScroll = 0;
  private dragging = false;
  private moved = false;
  private downX = 0;
  private downY = 0;
  private startScroll = 0;
  constructor(
    w: number,
    h: number,
    draws: PackDraw[],
    flipped: boolean[],
    prevOwned: Partial<Record<CardId, number>>,
    cb: RevealCallbacks,
  ) {
    super();
    this.w = w;
    this.h = h;
    this.draws = draws;
    this.flipped = flipped;
    this.prevOwned = prevOwned;
    this.cb = cb;
    this.ten = draws.length > 5;
    if (this.ten) {
      const cols = 10;
      const gap = 6;
      const cardW = (this.w - 16 - (cols - 1) * gap) / cols;
```

后段 minigame/src/ui/shop.ts:219 至 minigame/src/ui/shop.ts:275

```
    const rows = Math.ceil(draws.length / cols);
    const contentH = rows * (cardW * 1.5 + gap) + 8;
    this.maxScroll = Math.max(0, contentH - this.h);
  }
  loadCardImage(assetUrl('art/card-back-v1.webp'))
    .then((img) => {
      this.backImg = img;
    })
    .catch(() => {});
}
private isNew(d: PackDraw): boolean {
  return !d.converted && (this.prevOwned[d.id] ?? 0) === 0;
}
// ── Geometry ──
private fanCardW(): number {
  return clamp(96, this.w * 0.14, 130);
}
/** Returns draw rect + rotation for fan mode, or null if offscreen. */
private fanRect(i: number): { x: number; y: number; w: number; h: number; rot: number } {
  const cw = this.fanCardW();
  const ch = cw * 1.5;
  const fanX = clamp(34, this.w * 0.12, 64);
  const cx = this.w / 2;
  const cy = this.h * 0.42;
  const n = this.draws.length;
  const mid = (n - 1) / 2;
  const rot = (i - mid) * 11 * (Math.PI / 180);
  const spread = fanX + cw * 0.62;
  const x = cx + (i - mid) * spread - cw / 2;
  const y = cy + Math.abs(i - mid) * 12 - ch / 2;
  return { x, y, w: cw, h: ch, rot };
}
private gridRect(i: number): { x: number; y: number; w: number; h: number } {
  const cols = 10;
  const gap = 6;
  const cw = (this.w - 16 - (cols - 1) * gap) / cols;
  const ch = cw * 1.5;
  const col = i % cols;
  const row = Math.floor(i / cols);
  return {
    x: 8 + col * (cw + gap),
    y: 8 + row * (ch + gap) - this.scrollY,
    w: cw,
    h: ch,
  };
}
// ── Drawing ──
protected drawSelf(ctx: CanvasRenderingContext2D): void {
  ctx.fillStyle = COLORS.bg;
  ctx.fillRect(0, 0, this.w, this.h);
```

后段 minigame/src/ui/shop.ts:276 至 minigame/src/ui/shop.ts:328

```
    ctx.save();
    ctx.beginPath();
    ctx.rect(0, 0, this.w, this.h);
    ctx.clip();
    if (this.ten) this.drawGrid(ctx);
    else this.drawFan(ctx);
    ctx.restore();
  }
  private drawFan(ctx: CanvasRenderingContext2D): void {
    // Draw order: edges first, center last (on top).
    const n = this.draws.length;
    const order = [0, n - 1, 1, n - 2, 2].filter((i) => i >= 0 && i < n);
    const seen = new Set<number>();
    for (const i of order) {
      if (seen.has(i)) continue;
      seen.add(i);
      this.drawFanCard(ctx, i);
    }
    // Any remaining (n>5 safety)
    for (let i = 0; i < n; i++) {
      if (!seen.has(i)) this.drawFanCard(ctx, i);
    }
  }
  private drawFanCard(ctx: CanvasRenderingContext2D, i: number): void {
    const r = this.fanRect(i);
    const d = this.draws[i];
    ctx.save();
    ctx.translate(r.x + r.w / 2, r.y + r.h / 2);
    ctx.rotate(r.rot);
    if (!this.flipped[i]) {
      this.drawBack(ctx, -r.w / 2, -r.h / 2, r.w, r.h);
    } else {
      drawCardFace(ctx, d.id, -r.w / 2, -r.h / 2, r.w);
      this.drawBadges(ctx, d, -r.w / 2, -r.h / 2, r.w, r.h);
      // Rarity label under the card
      const label =
        RARITY_LABEL[d.rarity] +
        (this.isNew(d) ? '·新卡' : d.converted ? '·转化' : '');
      drawTextCentered(
        ctx,
        label,
        0,
        r.h / 2 + 10,
        9,
        RARITY_COLOR[d.rarity],
      );
    }
    ctx.restore();
  }
  private drawGrid(ctx: CanvasRenderingContext2D): void {
```

后段 minigame/src/ui/shop.ts:329 至 minigame/src/ui/shop.ts:380

```
    for (let i = 0; i < this.draws.length; i++) {
      const r = this.gridRect(i);
      if (r.y + r.h < 0 || r.y > this.h) continue;
      const d = this.draws[i];
      if (!this.flipped[i]) {
        this.drawBack(ctx, r.x, r.y, r.w, r.h);
      } else {
        drawCardFace(ctx, d.id, r.x, r.y, r.w);
        this.drawBadges(ctx, d, r.x, r.y, r.w, r.h);
      }
    }
  }
  private drawBack(
    ctx: CanvasRenderingContext2D,
    x: number,
    y: number,
    w: number,
    h: number,
  ): void {
    if (this.backImg) {
      ctx.drawImage(this.backImg, x, y, w, h);
    } else {
      ctx.fillStyle = '#2a332a';
      ctx.fillRect(x, y, w, h);
    }
    ctx.strokeStyle = COLORS.border;
    ctx.lineWidth = 1;
    ctx.strokeRect(x + 0.5, y + 0.5, w - 1, h - 1);
  }
  private drawBadges(
    ctx: CanvasRenderingContext2D,
    d: PackDraw,
    x: number,
    y: number,
    w: number,
    h: number,
  ): void {
    if (this.isNew(d)) {
      ctx.fillStyle = COLORS.red;
      roundRect(ctx, x + 2, y + 2, 20, 11, 2);
      ctx.fill();
      drawTextCentered(ctx, '新', x + 12, y + 8, 9, '#ffffff', 'bold');
    }
    if (d.converted) {
      ctx.fillStyle = COLORS.accentDark;
      const bw = 34;
      roundRect(ctx, x + w - bw - 2, y + 2, bw, 11, 2);
      ctx.fill();
      drawTextCentered(ctx, `+${d.gold}金`, x + w - bw / 2 - 2, y + 8, 8, '#ffffff');
    }
```

后段 minigame/src/ui/shop.ts:381 至 minigame/src/ui/shop.ts:436

```
    void h;
  }
  // ── Hit testing ──
  /** Returns the draw index at a point, or -1. */
  private hitIndex(px: number, py: number): number {
    if (this.ten) {
      for (let i = this.draws.length - 1; i >= 0; i--) {
        const r = this.gridRect(i);
        if (px >= r.x && px <= r.x + r.w && py >= r.y && py <= r.y + r.h) return i;
      }
      return -1;
    }
    // Fan: test in reverse draw order
    const n = this.draws.length;
    const order = [2, 3, 1, 4, 0].filter((i) => i >= 0 && i < n);
    for (const i of order) {
      const r = this.fanRect(i);
      // Inverse-rotate the point around the card center
      const cx = r.x + r.w / 2;
      const cy = r.y + r.h / 2;
      const dx = px - cx;
      const dy = py - cy;
      const cos = Math.cos(-r.rot);
      const sin = Math.sin(-r.rot);
      const lx = dx * cos - dy * sin;
      const ly = dx * sin + dy * cos;
      if (Math.abs(lx) <= r.w / 2 && Math.abs(ly) <= r.h / 2) return i;
    }
    return -1;
  }
  // ── Touch ──
  onTouchDown(x: number, y: number): boolean {
    this.dragging = true;
    this.moved = false;
    this.downX = x;
    this.downY = y;
    this.startScroll = this.scrollY;
    return true;
  }
  onTouchMove(x: number, y: number): void {
    if (!this.dragging) return;
    const dx = x - this.downX;
    const dy = y - this.downY;
    if (Math.abs(dx) > 8 || Math.abs(dy) > 8) this.moved = true;
    if (this.ten && this.moved) {
      this.scrollY = clamp(this.startScroll - dy, 0, this.maxScroll);
    }
  }
  onTouchUp(x: number, y: number): void {
    if (!this.dragging) return;
```

后段 minigame/src/ui/shop.ts:437 至 minigame/src/ui/shop.ts:494

```
    this.dragging = false;
    if (this.moved) return;
    const idx = this.hitIndex(x, y);
    if (idx >= 0) {
      if (!this.flipped[idx]) {
        this.flipped[idx] = true;
        this.cb.onFlip();
      } else {
        this.cb.onDetail(this.draws[idx].id);
      }
    } else {
      this.cb.onFlipAll();
    }
  }
}
// ── Shop screen ──────────────────────────────────────────────────────────
type Phase = 'idle' | 'tearing' | 'fanned';
export class ShopScreen extends Screen {
  private router: Router;
  private phase: Phase = 'idle';
  private result: PackResult | null = null;
  private prevOwned: Partial<Record<CardId, number>> = {};
  private flipped: boolean[] = [];
  private message = '';
  private messageAt = 0;
  private adReadyAt = 0;
  private packImg: HTMLImageElement | null = null;
  private tearWidget: TearWidget | null = null;
  private revealWidget: RevealAreaWidget | null = null;
  private ownedList: ScrollList | null = null;
  private btnPack: Button | null = null;
  private btnTen: Button | null = null;
  private btnAd: Button | null = null;
  private bottomButtons: Button[] = [];
  constructor(screenW: number, screenH: number, router: Router) {
    super(screenW, screenH);
    this.router = router;
    try {
      const saved = Number(localStorage.getItem(AD_COOLDOWN_STORAGE) ?? '0');
      this.adReadyAt = Number.isFinite(saved) ? saved : 0;
      if (this.adReadyAt < Date.now()) this.adReadyAt = 0;
    } catch {
      this.adReadyAt = 0;
    }
    loadCardImage(assetUrl('art/card-pack-v1.webp'))
      .then((img) => {
        this.packImg = img;
      })
      .catch(() => {});
  }
```

后段 minigame/src/ui/shop.ts:496 至 minigame/src/ui/shop.ts:553

```
  onEnter(): void {
    this.buildIdle();
  }
  // ── Idle layout ──
  private buildIdle(): void {
    this.phase = 'idle';
    this.clearChildren();
    this.result = null;
    this.bottomButtons = [];
    // Top bar
    const back = new Button('返回', 64, 28);
    back.x = 8;
    back.y = 6;
    back.onTap = () => this.router.navigate('lobby');
    this.addChild(back);
    // Left column panel content
    const btnW = 168;
    const btnX = 8 + (LEFT_W - btnW) / 2;
    let btnY = TOP_H + 206;
    this.btnPack = new Button(`开一包 · ${PACK_COST}金`, btnW, 30);
    this.btnPack.x = btnX;
    this.btnPack.y = btnY;
    this.btnPack.onTap = () => this.startReveal(false);
    this.addChild(this.btnPack);
    btnY += 34;
    this.btnTen = new Button(`连开十包 · ${TEN_PACK_COST}金`, btnW, 30);
    this.btnTen.x = btnX;
    this.btnTen.y = btnY;
    this.btnTen.onTap = () => this.startReveal(true);
    this.addChild(this.btnTen);
    btnY += 34;
    this.btnAd = new Button('', btnW, 30);
    this.btnAd.x = btnX;
    this.btnAd.y = btnY;
    this.btnAd.onTap = () => this.watchAd();
    this.addChild(this.btnAd);
    // Right side: owned cards
    this.ownedList = new ScrollList(
      this.screenW - 240,
      this.screenH - 82,
      ROW_H,
    );
    this.ownedList.x = 232;
    this.ownedList.y = 74;
    this.ownedList.onItemTap = (index) => {
      const rows = this.ownedRows();
      const row = rows[index];
      if (!row) return;
      this.showDialog(new CardDetailDialog(row.id, () => this.closeDialog()));
    };
```

后段 minigame/src/ui/shop.ts:554 至 minigame/src/ui/shop.ts:609

```
    this.addChild(this.ownedList);
    this.refreshIdle();
  }
  private ownedRows(): OwnedRow[] {
    const col = lobbyState.collection;
    const ids = (Object.keys(CARDS) as CardId[]).filter(
      (id) => !CARDS[id].internal && ownedCount(col, id) > 0,
    );
    ids.sort((a, b) => {
      const ca = CARDS[a];
      const cb = CARDS[b];
      if (ca.cost !== cb.cost) return ca.cost - cb.cost;
      return ca.name.localeCompare(cb.name, 'zh-CN');
    });
    return ids.map((id) => new OwnedRow(id, ownedCount(col, id)));
  }
  private refreshIdle(): void {
    const col = lobbyState.collection;
    this.btnPack!.disabled = col.gold < PACK_COST;
    this.btnTen!.disabled = col.gold < TEN_PACK_COST;
    this.refreshAdButton();
    if (this.ownedList) {
      this.ownedList.setItems(this.ownedRows());
    }
  }
  private refreshAdButton(): void {
    if (!this.btnAd) return;
    const col = lobbyState.collection;
    const remaining = this.adReadyAt - Date.now();
    if (remaining > 0) {
      this.btnAd.label = `广告冷却 ${Math.ceil(remaining / 1000)}s`;
      this.btnAd.disabled = true;
    } else {
      this.btnAd.label = `看广告领 ${AD_REWARD} 金`;
      this.btnAd.disabled = false;
    }
    void col;
  }
  // ── Pack opening ──
  private startReveal(ten: boolean): void {
    const col = lobbyState.collection;
    try {
      const r = ten ? openTenPacks(col) : openPack(col);
      this.prevOwned = { ...col.owned };
      lobbyState.updateCollection(r.state);
      this.result = r;
      this.flipped = r.drawn.map(() => false);
      this.buildTearing();
    } catch (e) {
      this.message = e instanceof Error ? e.message : String(e);
```

后段 minigame/src/ui/shop.ts:610 至 minigame/src/ui/shop.ts:664

```
      this.messageAt = Date.now();
    }
  }
  private buildTearing(): void {
    this.phase = 'tearing';
    this.clearChildren();
    this.tearWidget = new TearWidget(this.screenW, this.screenH);
    this.tearWidget.onDone = () => this.buildFanned();
    this.addChild(this.tearWidget);
  }
  private buildFanned(): void {
    this.phase = 'fanned';
    this.clearChildren();
    this.tearWidget = null;
    if (!this.result) {
      this.buildIdle();
      return;
    }
    this.revealWidget = new RevealAreaWidget(
      this.screenW,
      this.screenH - BOTTOM_H,
      this.result.drawn,
      this.flipped,
      this.prevOwned,
      {
        onFlip: () => this.refreshBottomBar(),
        onFlipAll: () => {
          for (let i = 0; i < this.flipped.length; i++) this.flipped[i] = true;
          this.refreshBottomBar();
        },
        onDetail: (id) => {
          this.showDialog(new CardDetailDialog(id, () => this.closeDialog()));
        },
      },
    );
    this.addChild(this.revealWidget);
    this.refreshBottomBar();
  }
  private get allFlipped(): boolean {
    return this.flipped.every((f) => f);
  }
  private refreshBottomBar(): void {
    for (const b of this.bottomButtons) this.removeChild(b);
    this.bottomButtons = [];
    if (this.phase !== 'fanned' || !this.result) return;
    const y = this.screenH - 62;
    const h = 30;
    let x = 8;
    const add = (label: string, w: number, onTap: () => void, disabled = false) => {
      const b = new Button(label, w, h);
```

后段 minigame/src/ui/shop.ts:665 至 minigame/src/ui/shop.ts:718

```
      b.x = x;
      b.y = y;
      b.onTap = onTap;
      b.disabled = disabled;
      this.addChild(b);
      this.bottomButtons.push(b);
      x += w + 6;
    };
    if (!this.allFlipped) {
      add('全部翻开', 84, () => {
        for (let i = 0; i < this.flipped.length; i++) this.flipped[i] = true;
        this.refreshBottomBar();
      });
    }
    add('收下', 72, () => this.buildIdle(), !this.allFlipped);
    add(
      '再开一包',
      92,
      () => this.startReveal(false),
      lobbyState.collection.gold < PACK_COST,
    );
    add(
      '再开十包',
      92,
      () => this.startReveal(true),
      lobbyState.collection.gold < TEN_PACK_COST,
    );
  }
  private summaryText(): string {
    if (!this.result) return '';
    const newCount = this.result.drawn.filter(
      (d) => !d.converted && (this.prevOwned[d.id] ?? 0) === 0,
    ).length;
    let parts = [`新卡 ${newCount} 张`];
    if (this.result.goldGained > 0) {
      parts.push(`溢出转化 +${this.result.goldGained} 金币`);
    }
    if (this.result.pity) {
      parts.push('保底王牌已触发');
    }
    return parts.join(' · ');
  }
  // ── Ads ──
  private watchAd(): void {
    if (this.adReadyAt > Date.now()) return;
    lobbyState.updateCollection(grantAdReward(lobbyState.collection));
    this.adReadyAt = Date.now() + AD_COOLDOWN_MS;
    try {
      localStorage.setItem(AD_COOLDOWN_STORAGE, String(this.adReadyAt));
    } catch {
```

后段 minigame/src/ui/shop.ts:719 至 minigame/src/ui/shop.ts:774

```
      /* storage may be unavailable */
    }
    this.message = `广告奖励 +${AD_REWARD} 金币`;
    this.messageAt = Date.now();
    this.refreshIdle();
  }
  // ── Frame ──
  update(dt: number): void {
    if (this.phase === 'tearing' && this.tearWidget) {
      this.tearWidget.update(dt);
    } else if (this.phase === 'idle') {
      this.refreshAdButton();
    }
  }
  protected drawSelf(ctx: CanvasRenderingContext2D): void {
    ctx.fillStyle = COLORS.bg;
    ctx.fillRect(0, 0, this.screenW, this.screenH);
    if (this.phase === 'tearing') return;
    if (this.phase === 'idle') {
      this.drawIdle(ctx);
    } else if (this.phase === 'fanned') {
      drawTextCentered(
        ctx,
        this.summaryText(),
        this.screenW / 2,
        this.screenH - 86,
        12,
        COLORS.accent,
        'bold',
      );
      if (!this.allFlipped) {
        drawTextCentered(
          ctx,
          '点击卡片翻一张 · 点击桌面全翻',
          this.screenW / 2,
          this.screenH - 14,
          10,
          COLORS.textMuted,
        );
      }
    }
  }
  private drawIdle(ctx: CanvasRenderingContext2D): void {
    // Top bar title + gold
    drawTextCentered(ctx, '军需商店', this.screenW / 2, 20, 15, COLORS.text, 'bold');
    drawTextRight(
      ctx,
      `金币 ${lobbyState.collection.gold}`,
      this.screenW - 12,
      20,
```

后段 minigame/src/ui/shop.ts:775 至 minigame/src/ui/shop.ts:828

```
    13,
    COLORS.accent,
    'bold',
  );
  // Left panel background
  ctx.fillStyle = COLORS.bgPanel;
  roundRect(ctx, 8, TOP_H + 4, LEFT_W, this.screenH - TOP_H - 12, 6);
  ctx.fill();
  ctx.strokeStyle = COLORS.border;
  ctx.lineWidth = 1;
  ctx.stroke();
  // Pack image
  const packW = 100;
  const packH = clamp(this.screenH - 220, 90, 150);
  const packX = 8 + (LEFT_W - packW) / 2;
  const packY = TOP_H + 24;
  if (this.packImg) {
    ctx.drawImage(this.packImg, packX, packY, packW, packH);
  } else {
    ctx.fillStyle = COLORS.bgPanelLight;
    roundRect(ctx, packX, packY, packW, packH, 4);
    ctx.fill();
  }
  drawTextCentered(
    ctx,
    '前线卡包',
    8 + LEFT_W / 2,
    packY + packH + 14,
    12,
    COLORS.text,
  );
  // Rates
  const rateY = TOP_H + 206 + 34 * 3 + 40;
  drawTextCentered(
    ctx,
    `常规 ${(RARITY_RATE.common * 100).toFixed(0)}% · 精锐 ${(RARITY_RATE.rare * 100).toFixed(0)}%`,
    8 + LEFT_W / 2,
    rateY,
    9,
    COLORS.textMuted,
  );
  drawTextCentered(
    ctx,
    `王牌 ${(RARITY_RATE.epic * 100).toFixed(0)}% · 传奇 ${(RARITY_RATE.legendary * 100).toFixed(0)}%`,
    8 + LEFT_W / 2,
    rateY + 14,
    9,
    COLORS.textMuted,
  );
  // Message
```

后段 minigame/src/ui/shop.ts:829 至 minigame/src/ui/theme.ts:27

```
    if (this.message && Date.now() - this.messageAt < 4000) {
      drawTextCentered(
        ctx,
        this.message,
        8 + LEFT_W / 2,
        rateY + 40,
        11,
        COLORS.redLight,
      );
    }
    // Right side header
    const col = lobbyState.collection;
    const prog = collectionProgress(col);
    drawTextLeft(
      ctx,
      `已收集 ${prog.species}/${prog.total} 种 · 共 ${prog.copies} 张 · 已开 ${col.packsOpened} 包`,
      232,
      52,
      11,
      COLORS.textDim,
    );
    drawTextLeft(ctx, '我的卡牌', 232, 66, 12, COLORS.text, 'bold');
  }
}
/** Theme constants shared by all Canvas UI pages. */
export const COLORS = {
  bg: '#1a1f1a',
  bgPanel: '#232b23',
  bgPanelLight: '#2d3a2d',
  bgCard: '#2a332a',
  border: '#4a5a4a',
  borderLight: '#6a7a6a',
  text: '#e8e0d0',
  textDim: '#9a9a8a',
  textMuted: '#6a6a5a',
  accent: '#d4a843',
  accentDark: '#8a6a20',
  green: '#5a8a4a',
  greenLight: '#7aaa5a',
  red: '#b5453a',
  redLight: '#d4655a',
  blue: '#4a7ab5',
  blueLight: '#6a9ad4',
  gold: '#d4a843',
  hp: '#c44a3a',
  hpBg: '#3a2020',
  energy: '#d4a843',
  energyBg: '#3a3020',
  overlay: 'rgba(10, 12, 10, 0.85)',
  scrim: 'rgba(10, 12, 10, 0.6)',
```

后段 minigame/src/ui/theme.ts:28 至 minigame/src/ui/theme.ts:82

```
} as const;
export const FONT = {
  pixel: 'fusion-pixel',
  fallback: 'monospace',
} as const;
export const LAYOUT = {
  /** Standard margin from screen edge. */
  margin: 8,
  /** Standard widget gap. */
  gap: 6,
  /** Button height. */
  btnH: 36,
  /** Small button height. */
  btnSmH: 28,
  /** Panel corner radius. */
  radius: 4,
  /** Top bar height. */
  topBarH: 40,
  /** Bottom bar height. */
  bottomBarH: 120,
} as const;
export function font(size: number, weight: 'normal' | 'bold' = 'normal'): string {
  return `${weight === 'bold' ? 'bold ' : ''}${size}px "${FONT.pixel}", ${FONT.fallback}`;
}
/** Measure text width using the pixel font. */
export function measureText(
  ctx: CanvasRenderingContext2D,
  text: string,
  size: number,
  weight: 'normal' | 'bold' = 'normal',
): number {
  ctx.font = font(size, weight);
  return ctx.measureText(text).width;
}
/** Draw text centered horizontally at (cx, y). */
export function drawTextCentered(
  ctx: CanvasRenderingContext2D,
  text: string,
  cx: number,
  y: number,
  size: number,
  color: string = COLORS.text,
  weight: 'normal' | 'bold' = 'normal',
): void {
  ctx.font = font(size, weight);
  ctx.fillStyle = color;
  ctx.textAlign = 'center';
  ctx.textBaseline = 'middle';
  ctx.fillText(text, cx, y);
}
```

后段 minigame/src/ui/theme.ts:84 至 minigame/src/ui/theme.ts:135

```
/** Draw text left-aligned at (x, y). */
export function drawTextLeft(
  ctx: CanvasRenderingContext2D,
  text: string,
  x: number,
  y: number,
  size: number,
  color: string = COLORS.text,
  weight: 'normal' | 'bold' = 'normal',
): void {
  ctx.font = font(size, weight);
  ctx.fillStyle = color;
  ctx.textAlign = 'left';
  ctx.textBaseline = 'middle';
  ctx.fillText(text, x, y);
}
/** Draw text right-aligned at (x, y). */
export function drawTextRight(
  ctx: CanvasRenderingContext2D,
  text: string,
  x: number,
  y: number,
  size: number,
  color: string = COLORS.text,
  weight: 'normal' | 'bold' = 'normal',
): void {
  ctx.font = font(size, weight);
  ctx.fillStyle = color;
  ctx.textAlign = 'right';
  ctx.textBaseline = 'middle';
  ctx.fillText(text, x, y);
}
/** Draw a rounded rectangle path. */
export function roundRect(
  ctx: CanvasRenderingContext2D,
  x: number,
  y: number,
  w: number,
  h: number,
  r: number,
): void {
  const rr = Math.min(r, w / 2, h / 2);
  ctx.beginPath();
  ctx.moveTo(x + rr, y);
  ctx.lineTo(x + w - rr, y);
  ctx.arcTo(x + w, y, x + w, y + rr, rr);
  ctx.lineTo(x + w, y + h - rr);
  ctx.arcTo(x + w, y + h, x + w - rr, y + h, rr);
  ctx.lineTo(x + rr, y + h);
  ctx.arcTo(x, y + h, x, y + h - rr, rr);
```

后段 minigame/src/ui/theme.ts:136 至 minigame/src/ui/theme.ts:189

```
  ctx.lineTo(x, y + rr);
  ctx.arcTo(x, y, x + rr, y, rr);
  ctx.closePath();
}
/** Draw a filled panel with border. */
export function drawPanel(
  ctx: CanvasRenderingContext2D,
  x: number,
  y: number,
  w: number,
  h: number,
  fill: string = COLORS.bgPanel,
  border: string = COLORS.border,
  radius: number = LAYOUT.radius,
): void {
  roundRect(ctx, x, y, w, h, radius);
  ctx.fillStyle = fill;
  ctx.fill();
  ctx.strokeStyle = border;
  ctx.lineWidth = 1;
  ctx.stroke();
}
/** Draw a progress bar. */
export function drawBar(
  ctx: CanvasRenderingContext2D,
  x: number,
  y: number,
  w: number,
  h: number,
  pct: number,
  fillColor: string,
  bgColor: string = COLORS.hpBg,
): void {
  ctx.fillStyle = bgColor;
  ctx.fillRect(x, y, w, h);
  const fw = Math.max(0, Math.min(1, pct)) * w;
  if (fw > 0) {
    ctx.fillStyle = fillColor;
    ctx.fillRect(x, y, fw, h);
  }
  ctx.strokeStyle = COLORS.border;
  ctx.lineWidth = 1;
  ctx.strokeRect(x + 0.5, y + 0.5, w - 1, h - 1);
}
/** Clamp a value. */
export function clamp(v: number, lo: number, hi: number): number {
  return Math.max(lo, Math.min(hi, v));
}
/** Linear interpolate. */
export function lerp(a: number, b: number, t: number): number {
```

后段 minigame/src/ui/theme.ts:190 至 minigame/src/ui/theme.ts:241

```
  return a + (b - a) * t;
}
/**
 * Wrap text to fit a width. Returns lines. Handles CJK character-by-character
 * and Latin word-by-word wrapping.
 */
export function wrapText(
  ctx: CanvasRenderingContext2D,
  text: string,
  maxWidth: number,
  size: number,
  weight: 'normal' | 'bold' = 'normal',
): string[] {
  ctx.font = font(size, weight);
  const lines: string[] = [];
  const paragraphs = text.split('\n');
  for (const para of paragraphs) {
    let line = '';
    let i = 0;
    while (i < para.length) {
      const ch = para[i];
      // Latin word: grab until whitespace
      if (/[ -~]/.test(ch) && ch !== ' ') {
        let j = i;
        while (j < para.length && /[^\s一-□    -鿋□-□]/.test(para[j])) j++;
        const word = para.slice(i, j);
        const test = line ? line + word : word;
        if (ctx.measureText(test).width > maxWidth && line) {
          lines.push(line);
          line = word;
        } else {
          line = test;
        }
        i = j;
      } else {
        const test = line + ch;
        if (ctx.measureText(test).width > maxWidth && line) {
          lines.push(line);
          line = ch === ' ' ? '' : ch;
        } else {
          line = test;
        }
        i++;
      }
    }
    lines.push(line);
  }
  return lines;
}
/**
```

后段 minigame/src/ui/theme.ts:242 至 minigame/src/ui/touch-routing.ts:28

```
 * Draw wrapped text starting at (x, y). Returns the y position after the
 * last line.
 */
export function drawWrapped(
  ctx: CanvasRenderingContext2D,
  text: string,
  x: number,
  y: number,
  maxWidth: number,
  size: number,
  color: string = COLORS.text,
  weight: 'normal' | 'bold' = 'normal',
  lineHeight: number = size * 1.5,
): number {
  const lines = wrapText(ctx, text, maxWidth, size, weight);
  ctx.font = font(size, weight);
  ctx.fillStyle = color;
  ctx.textAlign = 'left';
  ctx.textBaseline = 'top';
  let cy = y;
  for (const line of lines) {
    ctx.fillText(line, x, cy);
    cy += lineHeight;
  }
  return cy;
}
import type { TouchEvent, TouchPoint } from './framework';
function points(list: any): TouchPoint[] {
  return Array.from(list ?? [], (t: any) => ({ id: t.identifier ?? 0, x: t.clientX, y: t.clientY }));
}
/** Keep current positions and distinguish interruption from deliberate release. */
export class TouchRouting {
  private active = new Map<number, TouchPoint>();
  private deliver: (event: TouchEvent) => void;
  constructor(deliver: (event: TouchEvent) => void) { this.deliver = deliver; }
  dispatch(type: TouchEvent['type'], raw: any): void {
    const changed = points(raw.changedTouches ?? raw.touches);
    for (const p of changed) {
      if (type === 'down' || this.active.has(p.id)) this.active.set(p.id, p);
    }
    this.deliver({ type, changed, points: [...this.active.values()] });
    if (type === 'up' || type === 'cancel')
      for (const p of changed) this.active.delete(p.id);
  }
  cancelAll(): void {
    const changed = [...this.active.values()];
    this.active.clear();
    this.deliver({ type: 'cancel', changed, points: [] });
  }
}
```